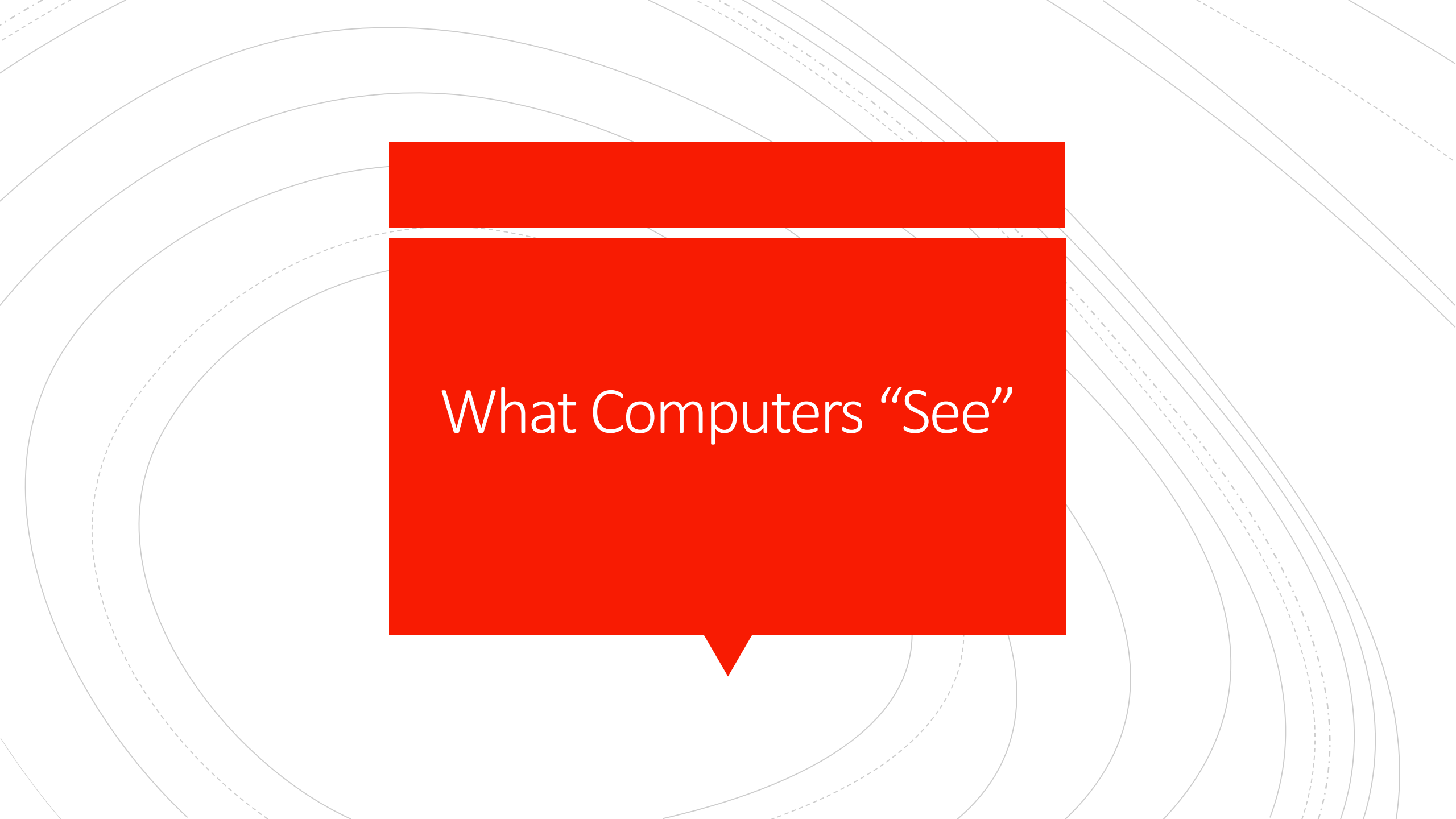


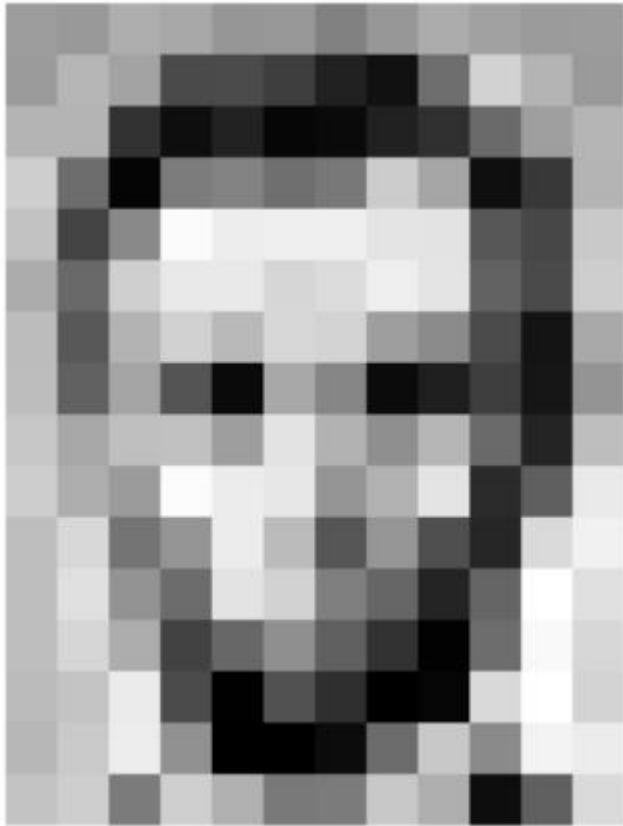
Deep Learning in Computer Vision

Bikash Santra
Senior Research Fellow
Electronics and Communication Sciences Unit
Indian Statistical Institute, Kolkata, India

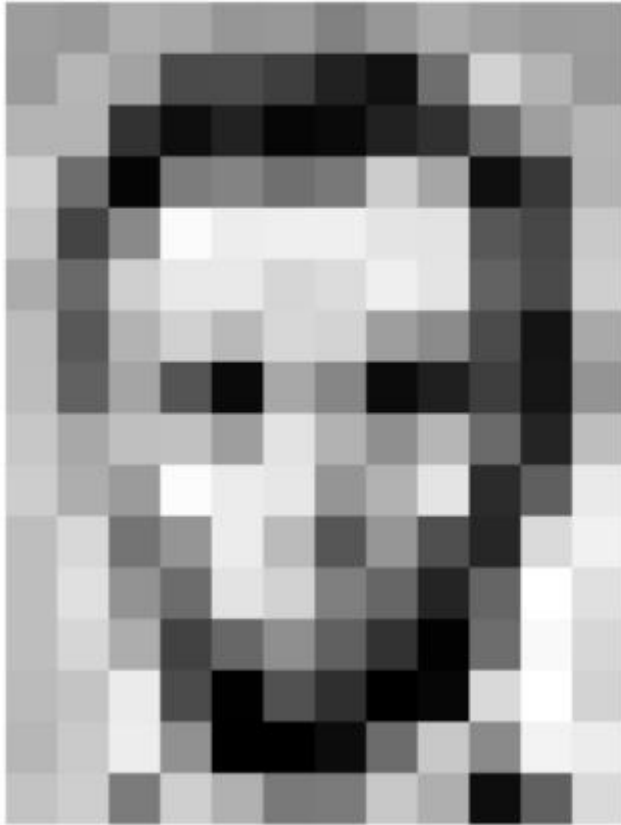
The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A bright red speech bubble is centered on the page, pointing downwards. The text "What Computers 'See'" is written in white inside the bubble.

What Computers “See”

Images Are Numbers

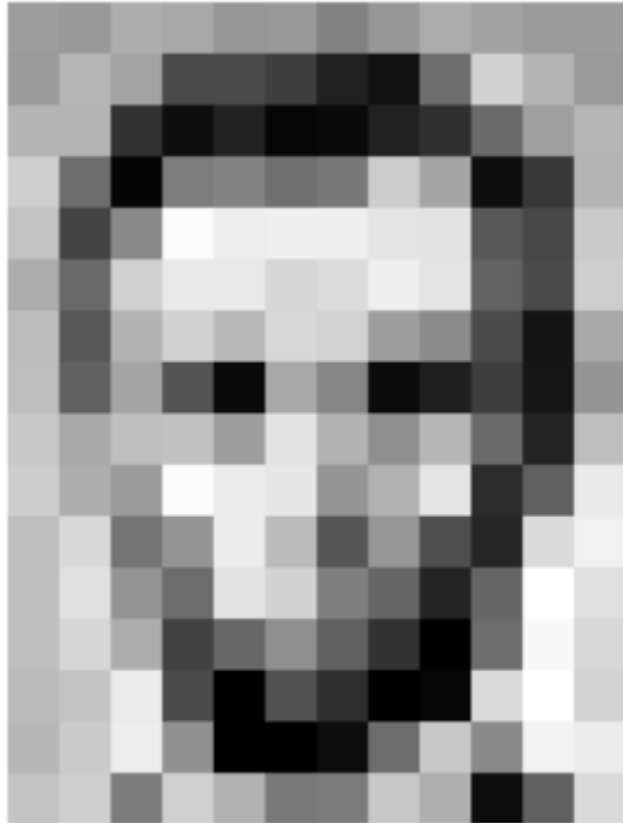


Images Are Numbers



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	6	124	131	111	120	204	166	15	56	180
194	68	137	251	237	299	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	85	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Images Are Numbers



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	94	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

What the computer sees

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	94	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

An image is just a matrix of numbers $[0,255]$!
i.e., $1080 \times 1080 \times 3$ for an RGB image

What is Image Processing?

1. Photometric Transformations

- a) Sharpening
- b) Smoothing
- c) Contrast enhancement
- d) Stretching

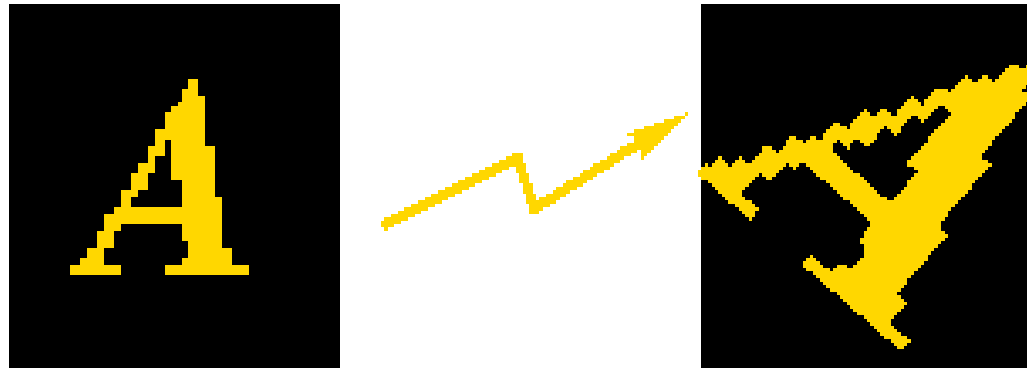


2. Geometric Transformations

- 1. Scaling
- 2. Rotations
- 3. Translation

3. Image Compression

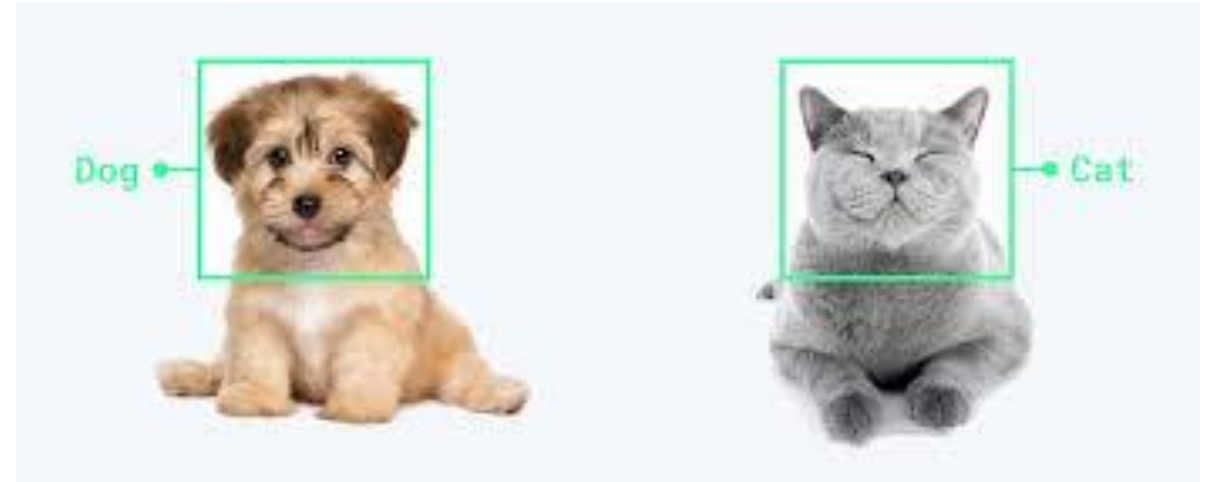
4. And many more



What is Computer Vision?

1. Emulates human vision
2. Goal is to understand images and its contents

Image classification



Semantic Segmentation



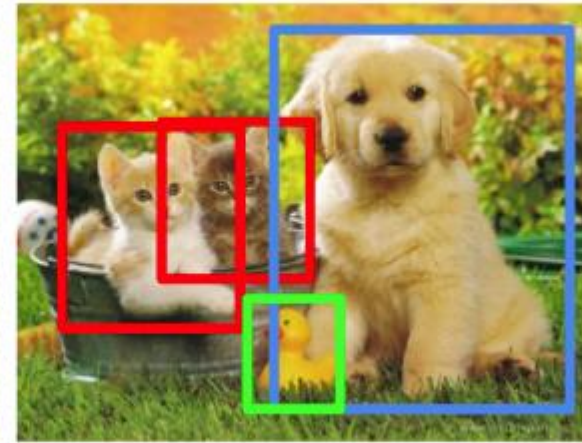
CAT

Classification + Localization



CAT

Object Detection



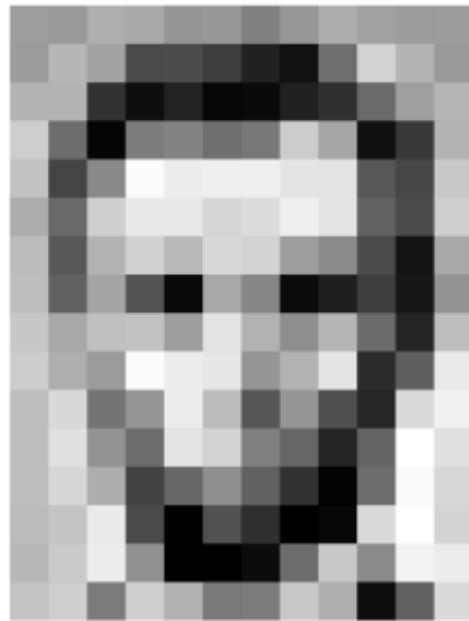
CAT, DOG, DUCK

Instance Segmentation



CAT, DOG, DUCK

Tasks in Computer Vision



Input Image



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	20								74	206
188	88	11								20	169
189	97	14								22	148
199	168	11								36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Pixel Representation

Feature
Extraction



classification

Lincoln

Washington

Jefferson

Obama

$$\begin{bmatrix} 0.8 \\ 0.1 \\ 0.05 \\ 0.05 \end{bmatrix}$$

- **Regression:** output variable takes continuous value
- **Classification:** output variable takes class label. Can produce probability of belonging to a particular class

High Level Feature Detection

Let's identify key features in each image category



Nose,
Eyes,
Mouth

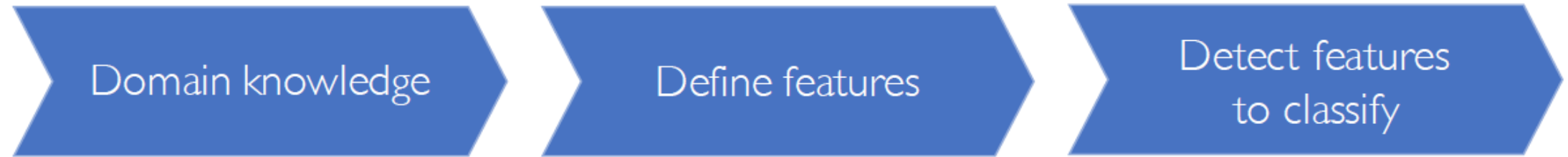


Wheels,
License Plate,
Headlights



Door,
Windows,
Steps

Manual Feature Extraction



Problems?

Manual Feature Extraction

Domain knowledge

Define features

Detect features
to classify

Viewpoint variation



Scale variation



Deformation



Occlusion



Illumination conditions



Background clutter



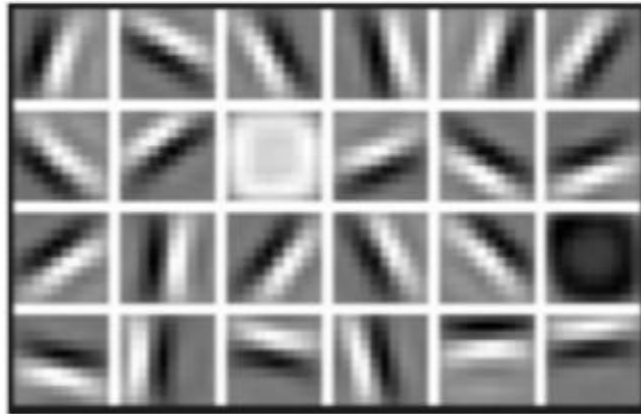
Intra-class variation



Hand engineered features are time consuming, brittle and not scalable in practice

Question 1 Can we learn the **underlying features** directly from data?

Low Level Features



Lines & Edges

Mid Level Features



Eyes & Nose & Ears

High Level Features

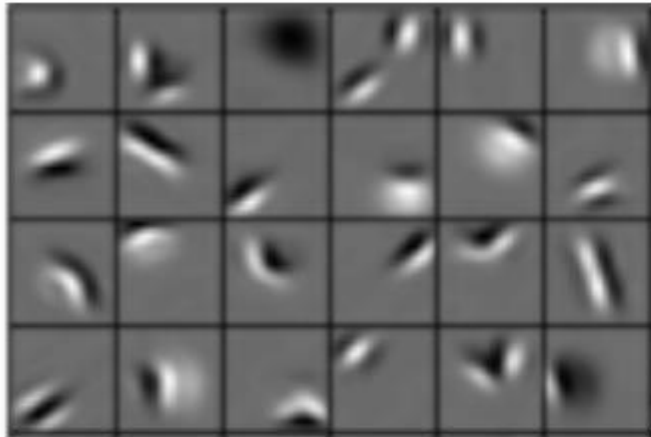


Facial Structure

Question 2

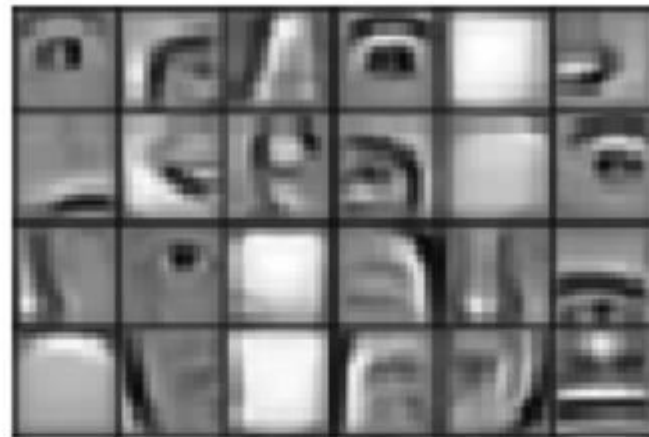
Can we **learn hierarchy of features** directly from the data instead of hand engineering?

Low level features



Edges, dark spots

Mid level features



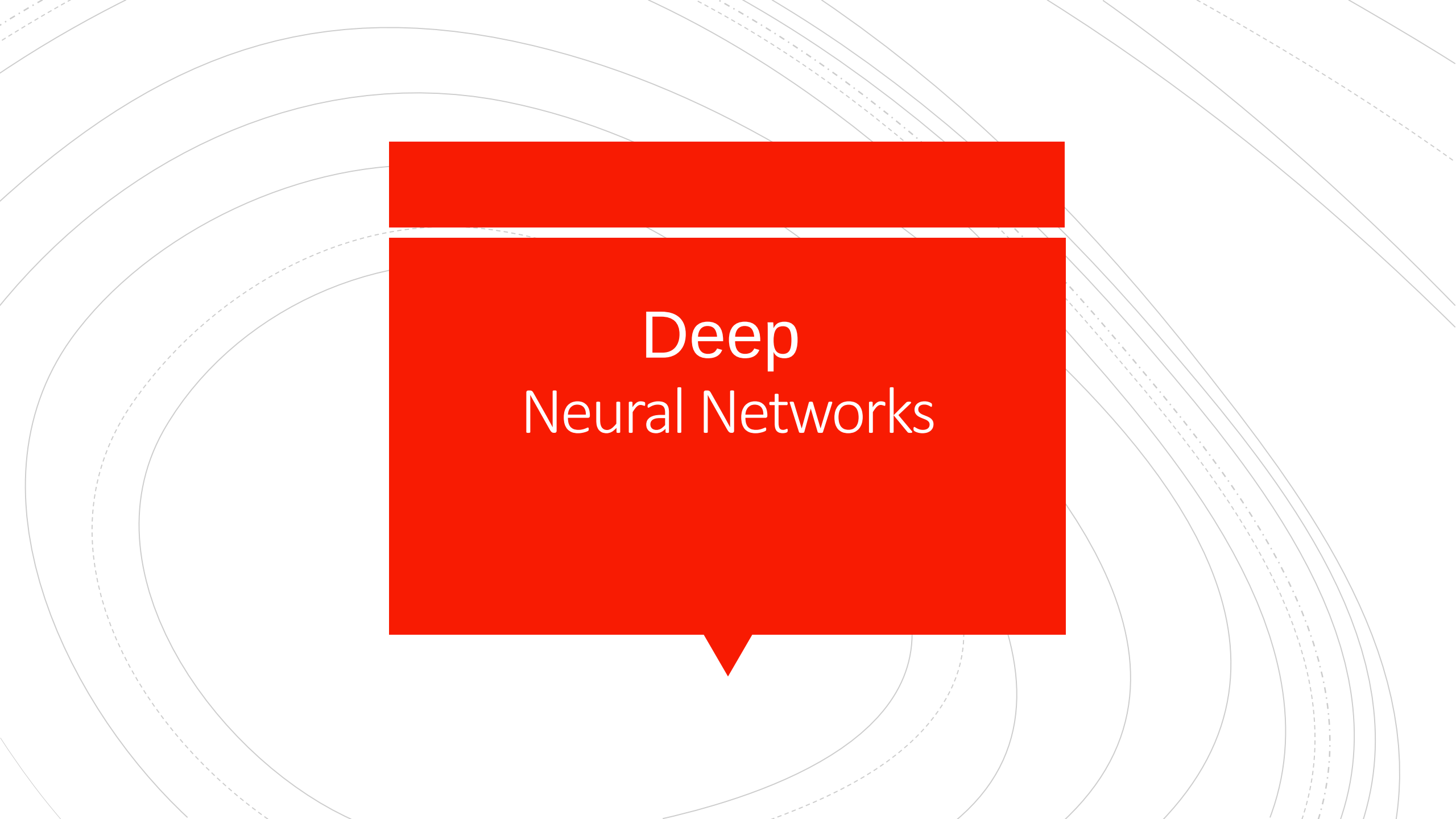
Eyes, ears, nose

High level features



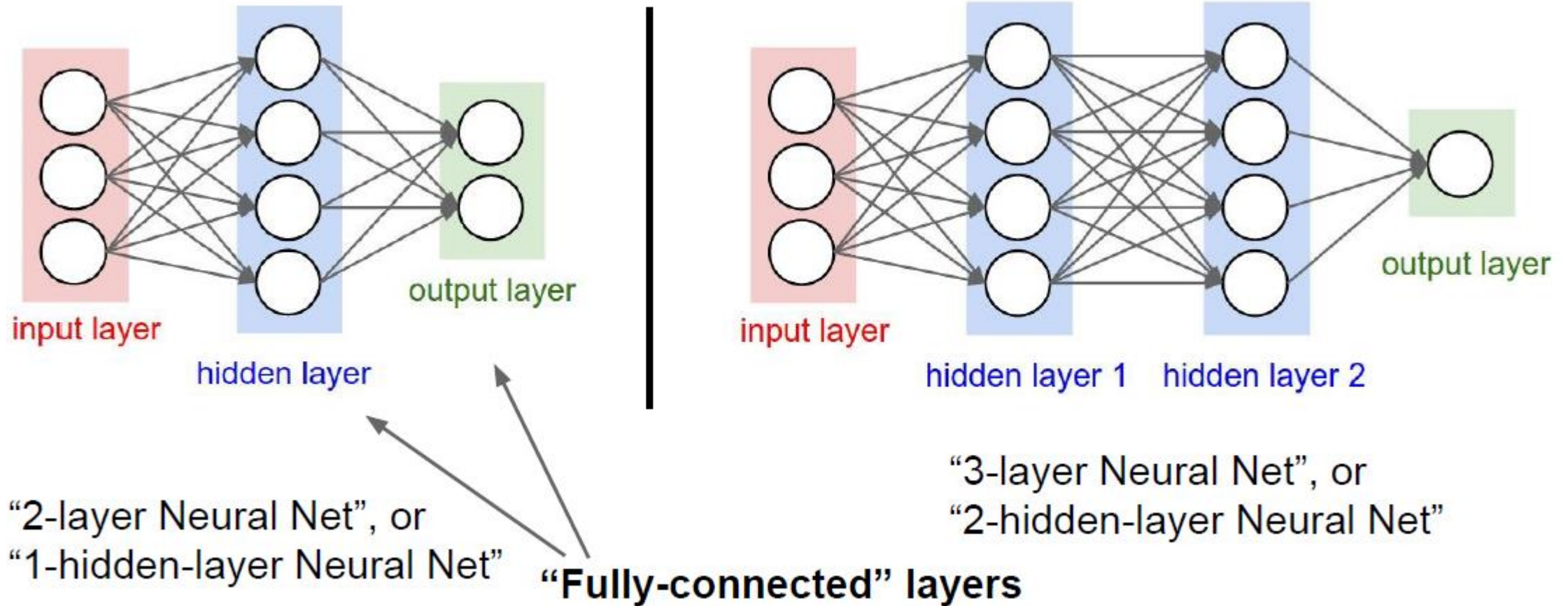
Facial structure

Yes, we can! Using deep learning

The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. In the center, there is a red speech bubble with a white border and a small tail pointing downwards.

Deep Neural Networks

Neural Networks: Architectures



Example Problem

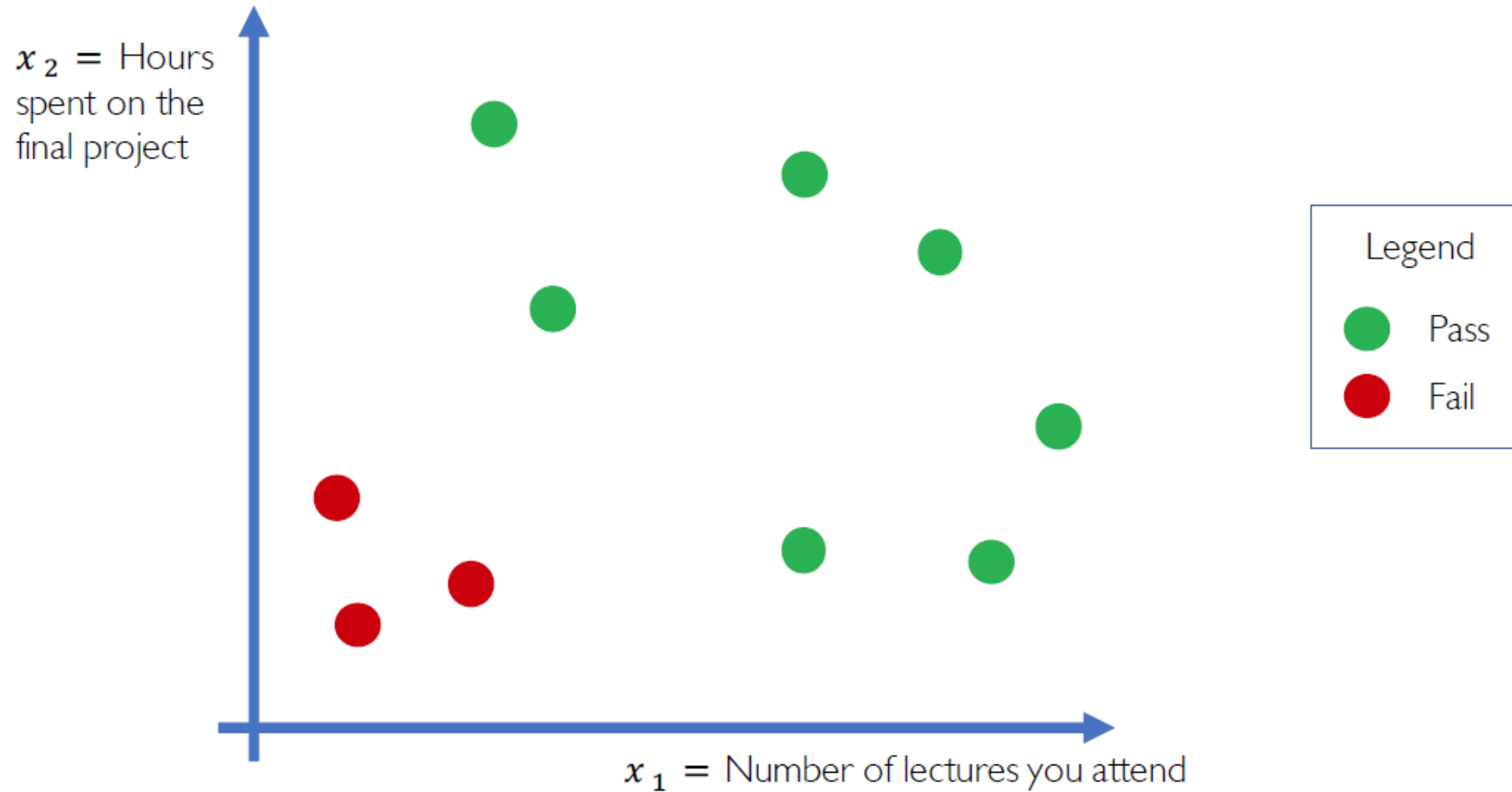
Will I pass this class?

Let's start with a simple two feature model

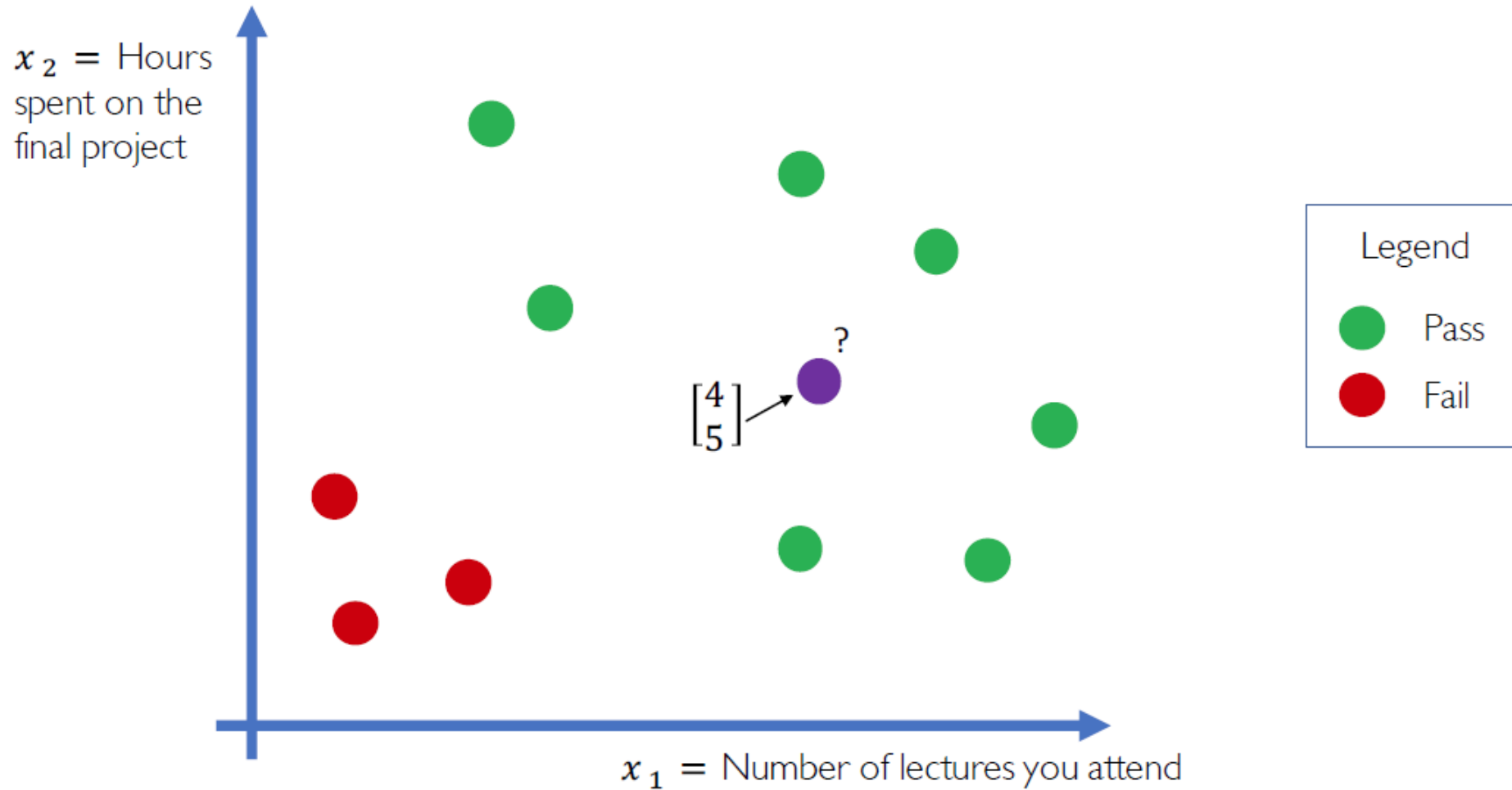
x_1 = Number of lectures you attend

x_2 = Hours spent on the final project

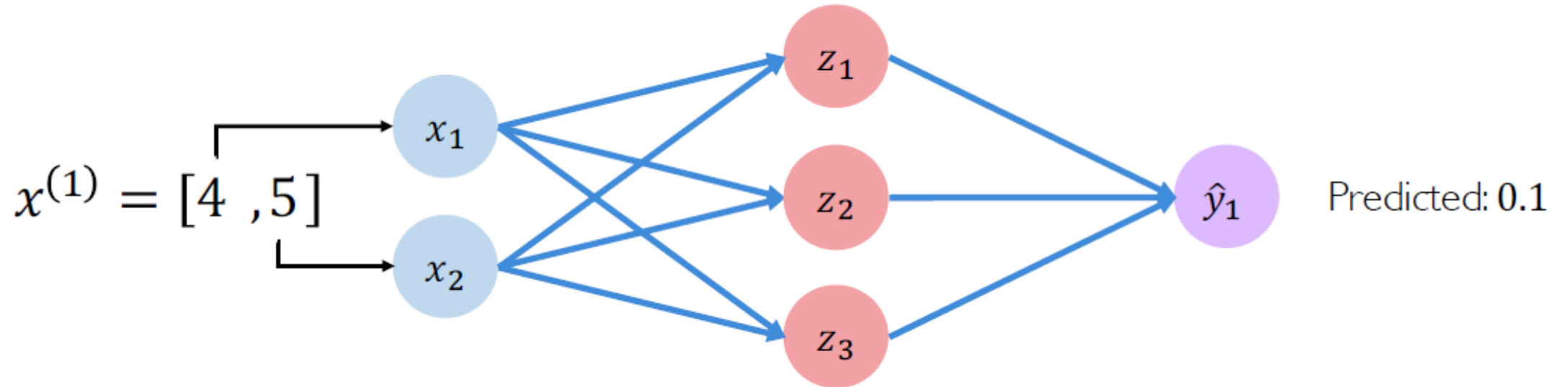
Example Problem: Will I pass this class?



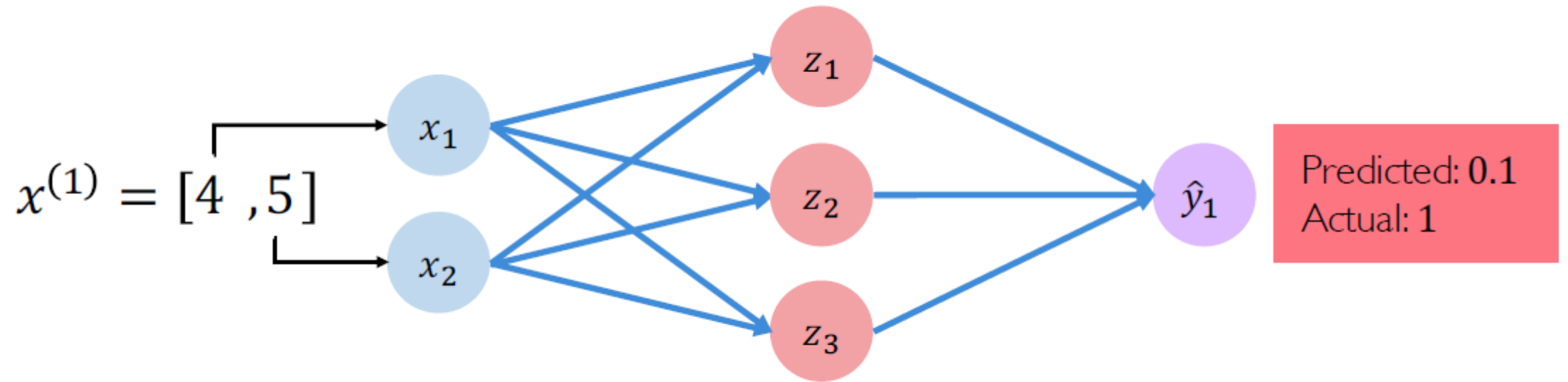
Example Problem: Will I pass this class?



Example Problem: Will I pass this class?

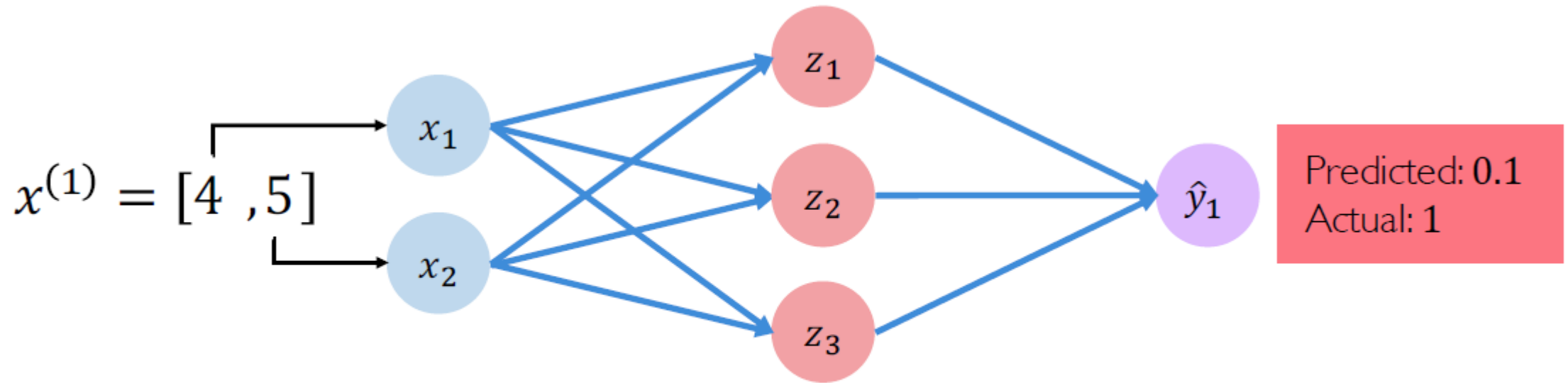


Example Problem: Will I pass this class?



Quantifying Loss

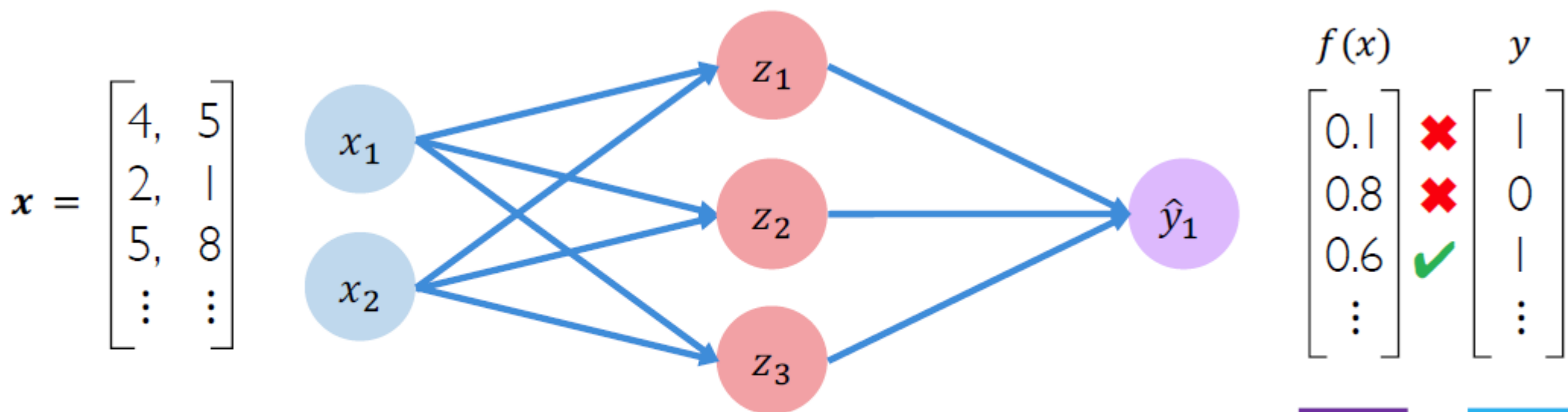
The **loss** of our network measures the cost incurred from incorrect predictions



$$\mathcal{L}(\underbrace{f(x^{(i)}; \boldsymbol{\theta})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Empirical Loss

The **empirical loss** measures the total loss over our entire dataset



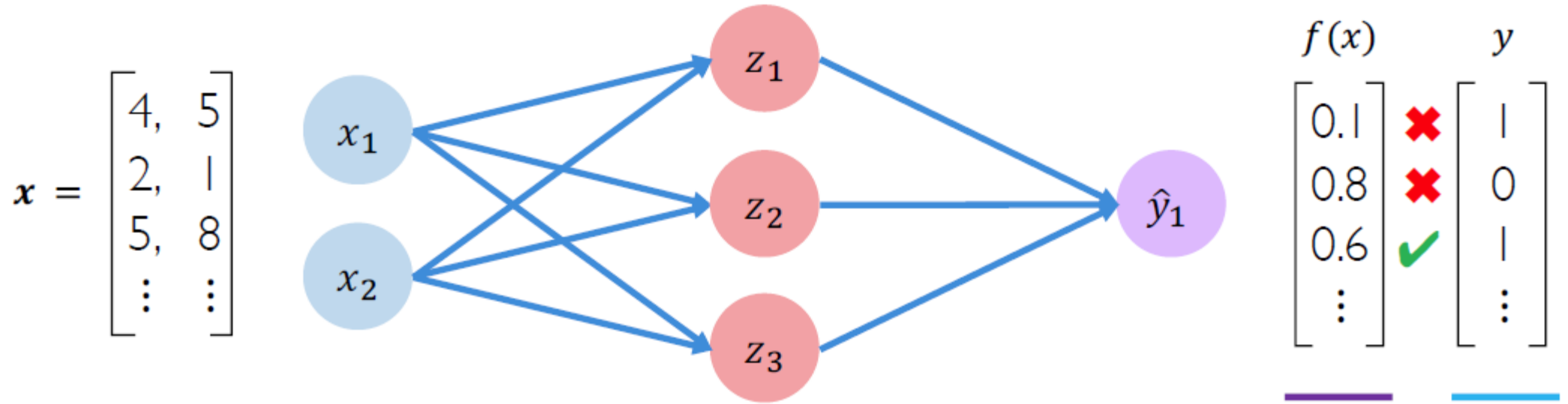
Also known as:

- Objective function
- Cost function
- Empirical Risk

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\underbrace{f(x^{(i)}; \theta)}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Binary Cross Entropy Loss

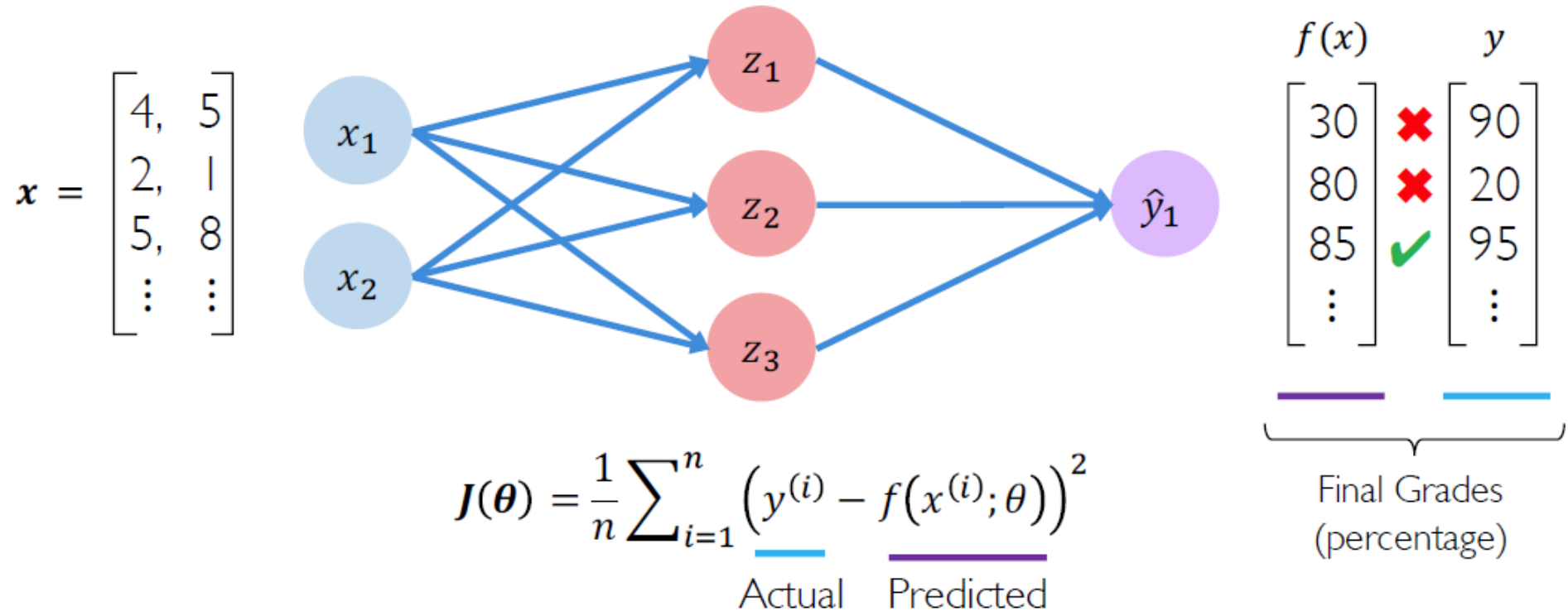
Cross entropy loss can be used with models that output a probability between 0 and 1



$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \underbrace{y^{(i)}}_{\text{Actual}} \log \left(\underbrace{f(x^{(i)}; \theta)}_{\text{Predicted}} \right) + (1 - \underbrace{y^{(i)}}_{\text{Actual}}) \log \left(1 - \underbrace{f(x^{(i)}; \theta)}_{\text{Predicted}} \right)$$

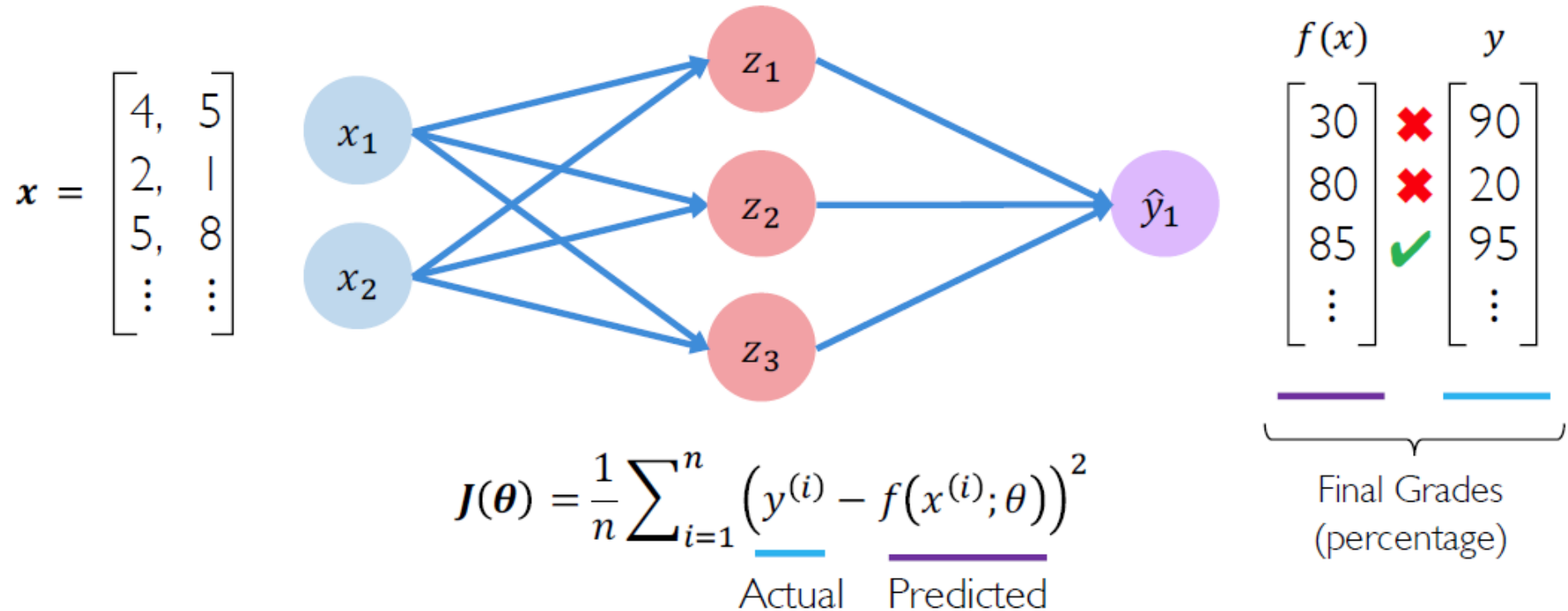
Mean Squared Error Loss

Mean squared error loss can be used with regression models that output continuous real numbers



Mean Squared Error Loss

Mean squared error loss can be used with regression models that output continuous real numbers



Loss Optimization

We want to find the network weights that **achieve the lowest loss**

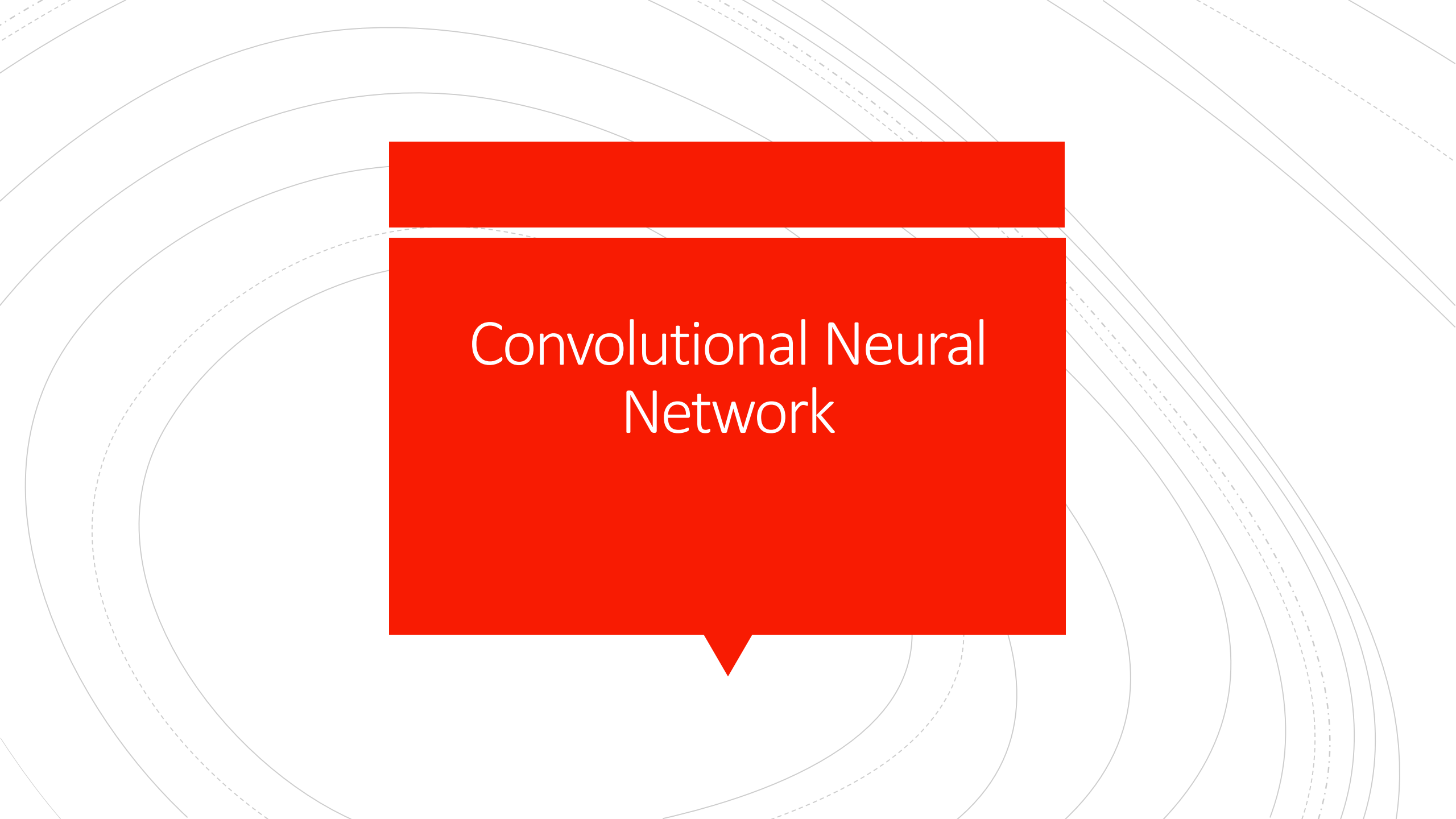
$$\theta^* = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \theta), y^{(i)})$$

$$\theta^* = \operatorname{argmin}_{\theta} J(\theta)$$

Solved using
gradient descent

Remember:
 $\theta = \{\theta^{(0)}, \theta^{(1)}, \dots\}$



The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A large red speech bubble is centered on the page, containing the text 'Convolutional Neural Network'.

Convolutional Neural Network

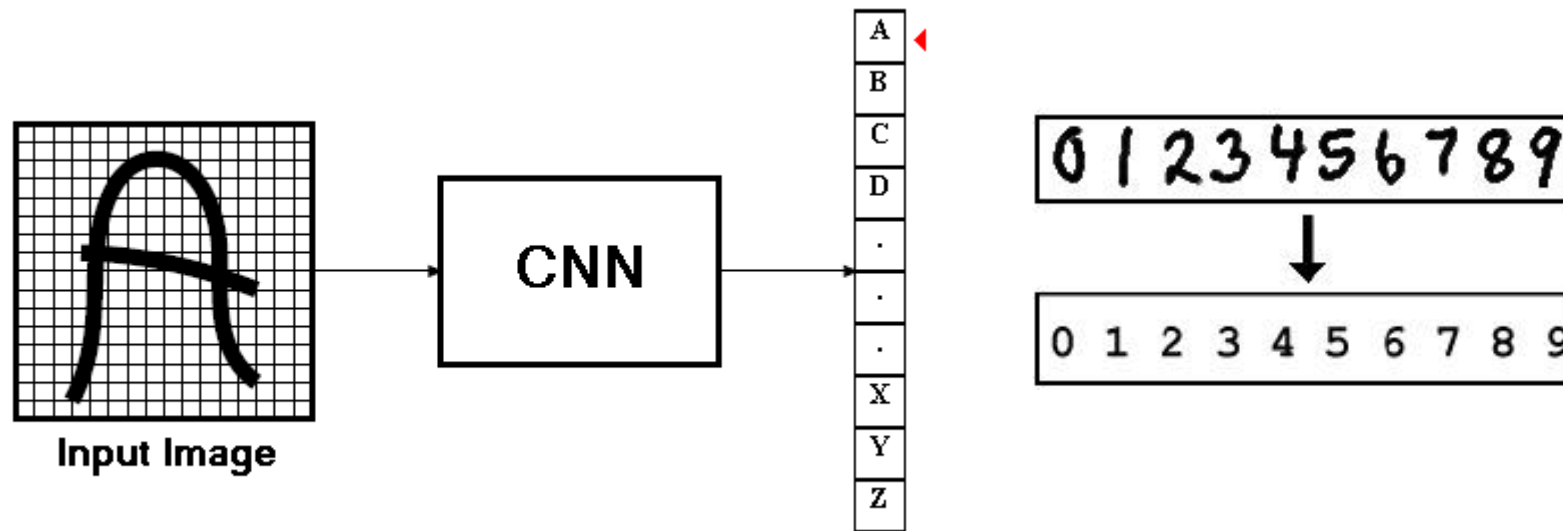
Introduction

🌀 Convolutional neural networks

- 🌀 Signal processing, Image processing

🌀 improvement over the multilayer perceptron

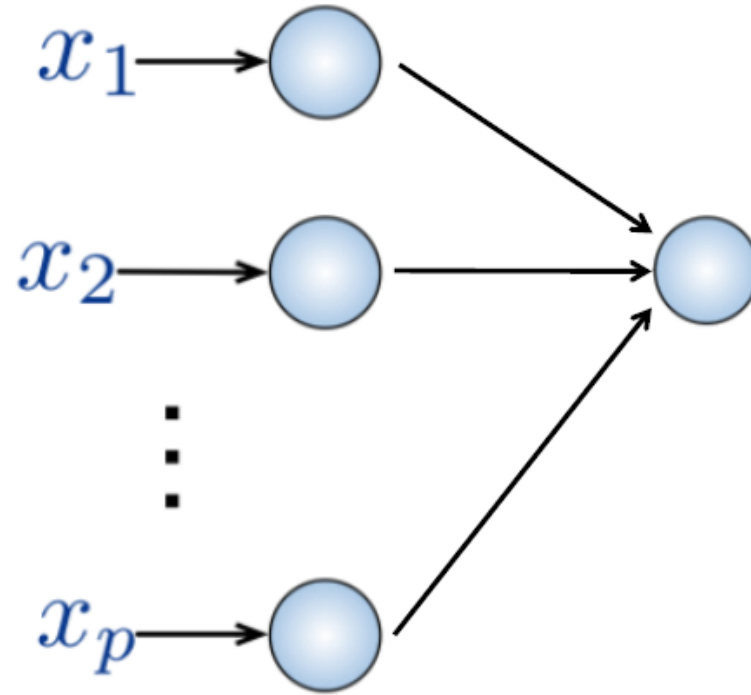
- 🌀 performance, accuracy and some degree of invariance to distortions in the input images



FCNN for Image Processing

Input:

- 2D image
- Vector of pixel values



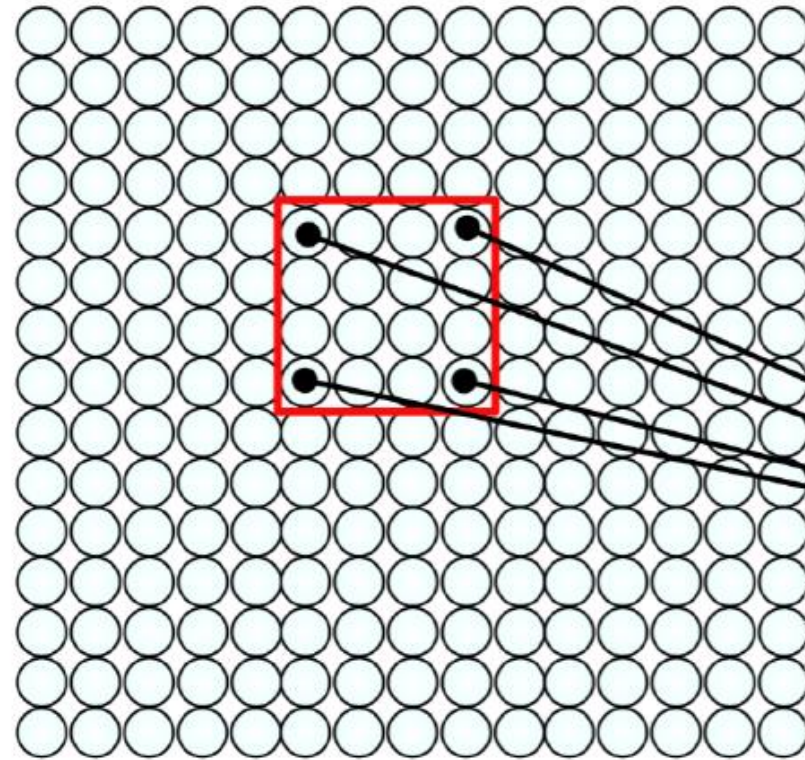
Fully connected:

- Connect neuron in hidden layer to all neurons in input layer
- No spatial information!
- And many, many parameters!

How can we use **spatial structure** in the input to inform the architecture of the network?

Using Spatial Structure

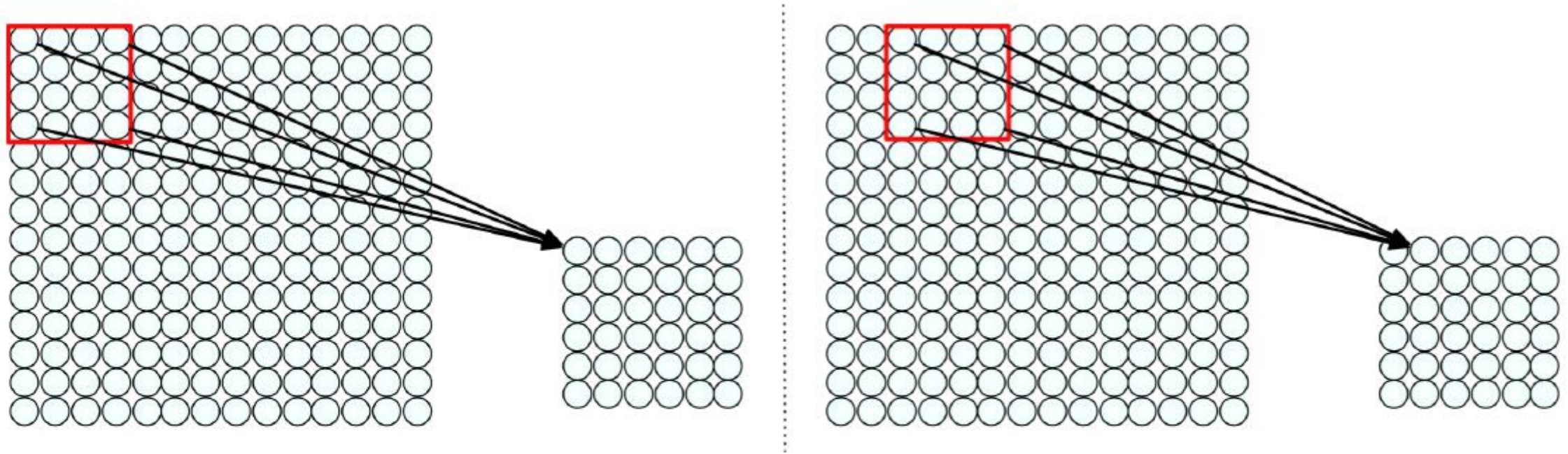
Input: 2D image.
Array of pixel values



Idea: connect patches of input to
neurons in hidden layer:

Neuron connected to region of
input. Only "sees" these values.

Using Spatial Structure



Connect patch in input layer to a single neuron in subsequent layer.

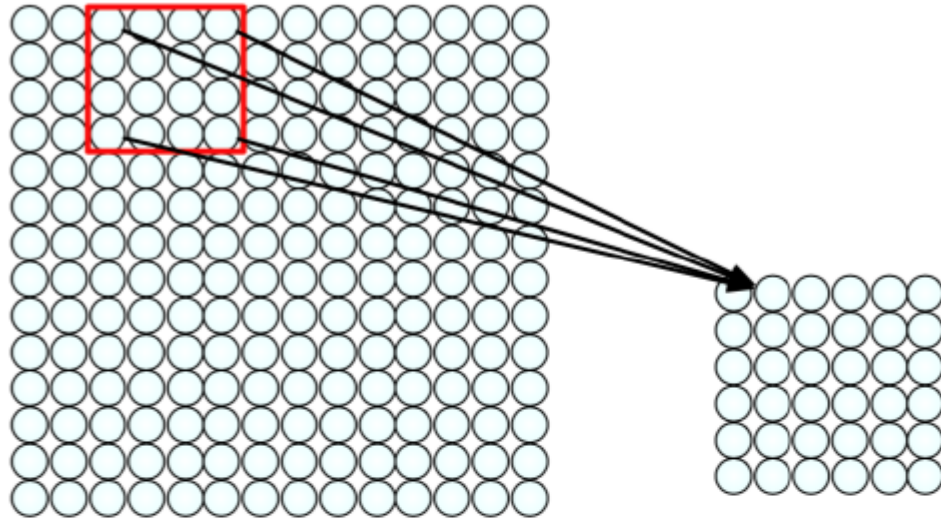
Use a sliding window to define connections.

*How can we **weight** the patch to detect particular features?*

Applying Filters to Extract Features

- 1) Apply a set of weights – a filter – to extract **local features**
- 2) Use **multiple filters** to extract different features
- 3) Spatially **share** parameters of each filter
(features that matter in one part of the input should matter elsewhere)

Feature Extraction with Convolution



- Filter of size 4x4 : 16 different weights
- Apply this same filter to 4x4 patches in input
- Shift by 2 pixels for next patch

This “patchy” operation is **convolution**

1) Apply a set of weights – a filter – to extract **local features**

2) Use **multiple filters** to extract different features

3) **Spatially share** parameters of each filter

Convolutional Neural Networks

- CNN is evolved basically to deal with images.

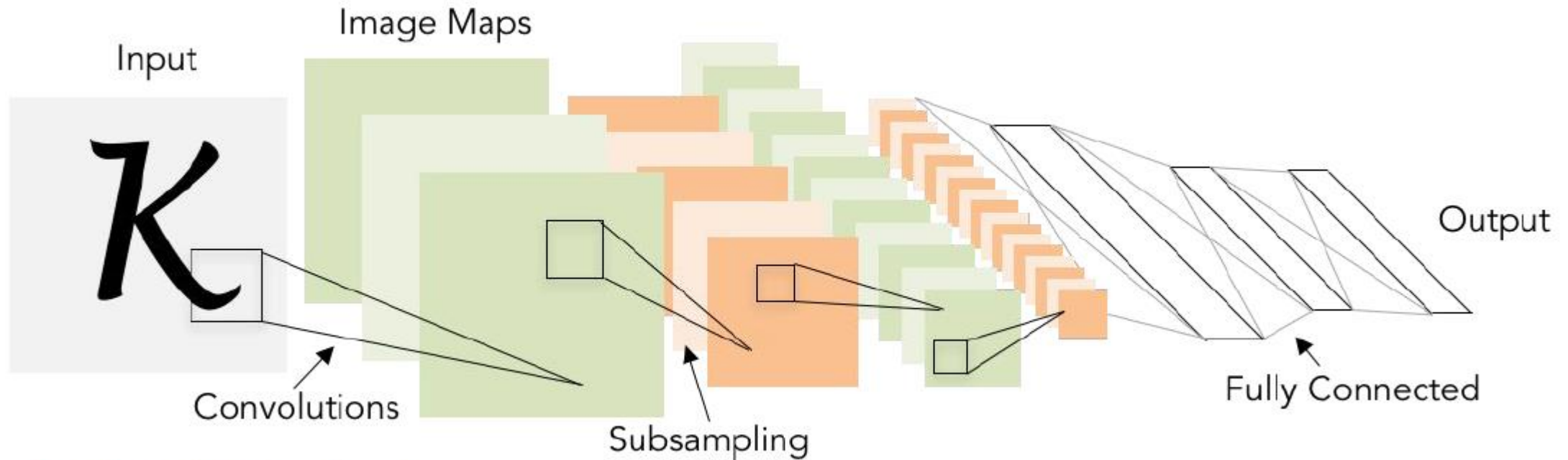
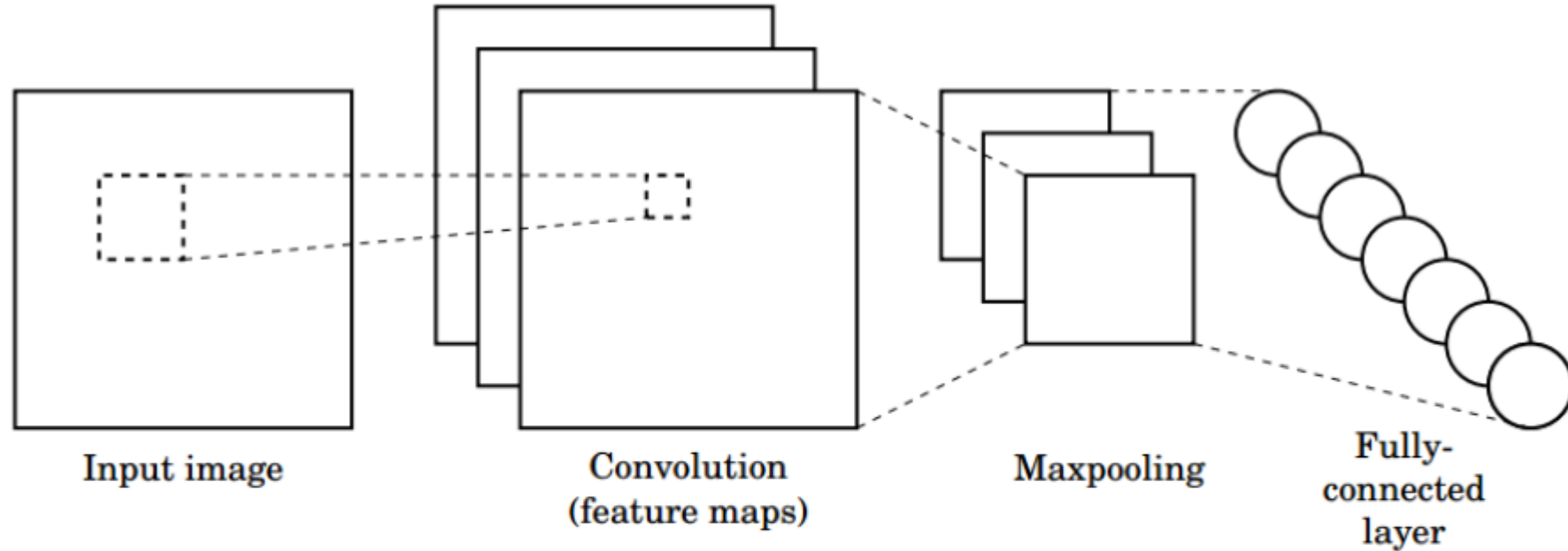


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

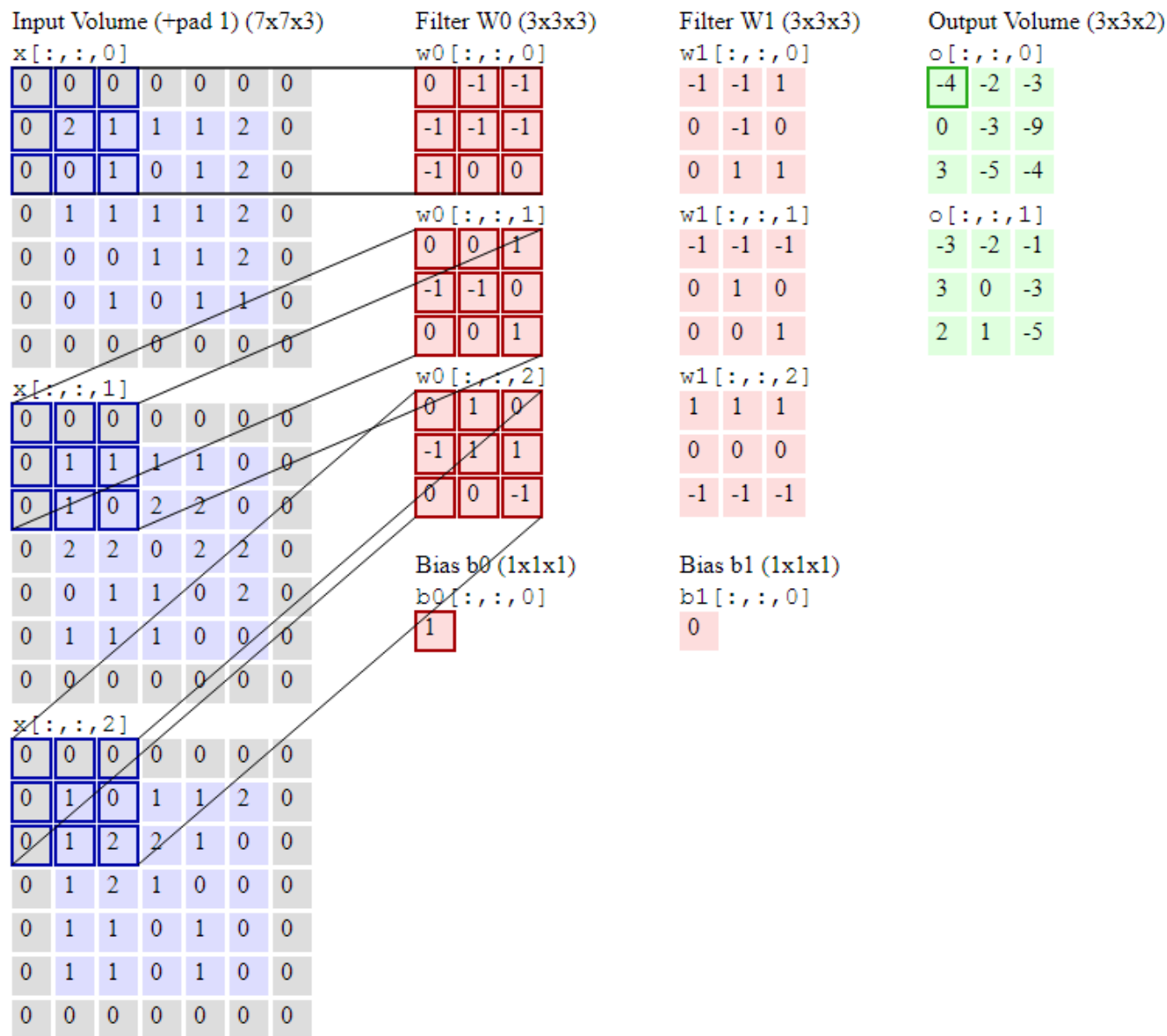
Components of CNN



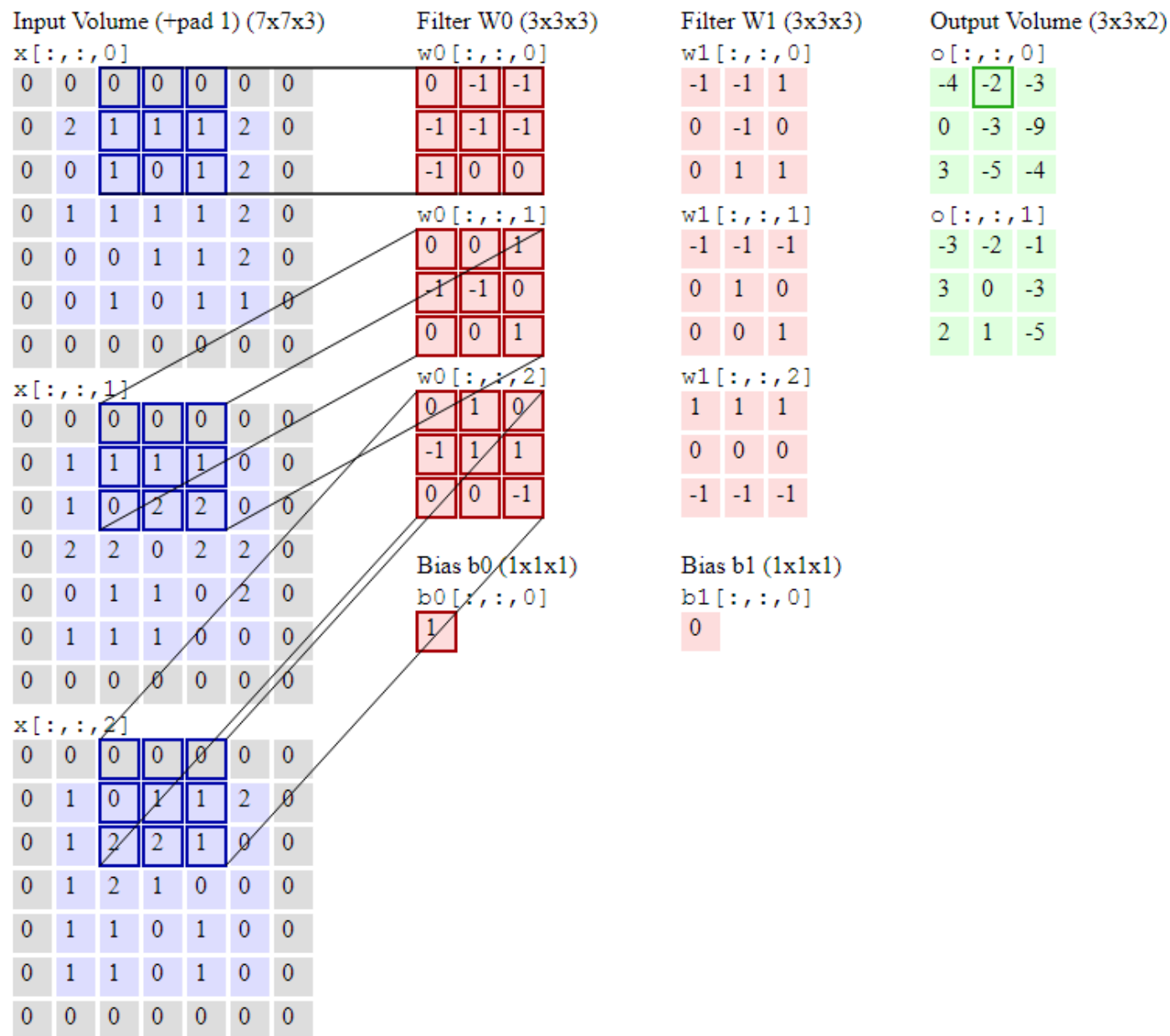
1. **Convolution:** Apply filters with learned weights to generate feature maps.
2. **Non-linearity:** Often ReLU.
3. **Pooling:** Downsampling operation on each feature map.

Train model with image data.
Learn weights of filters in convolutional layers.

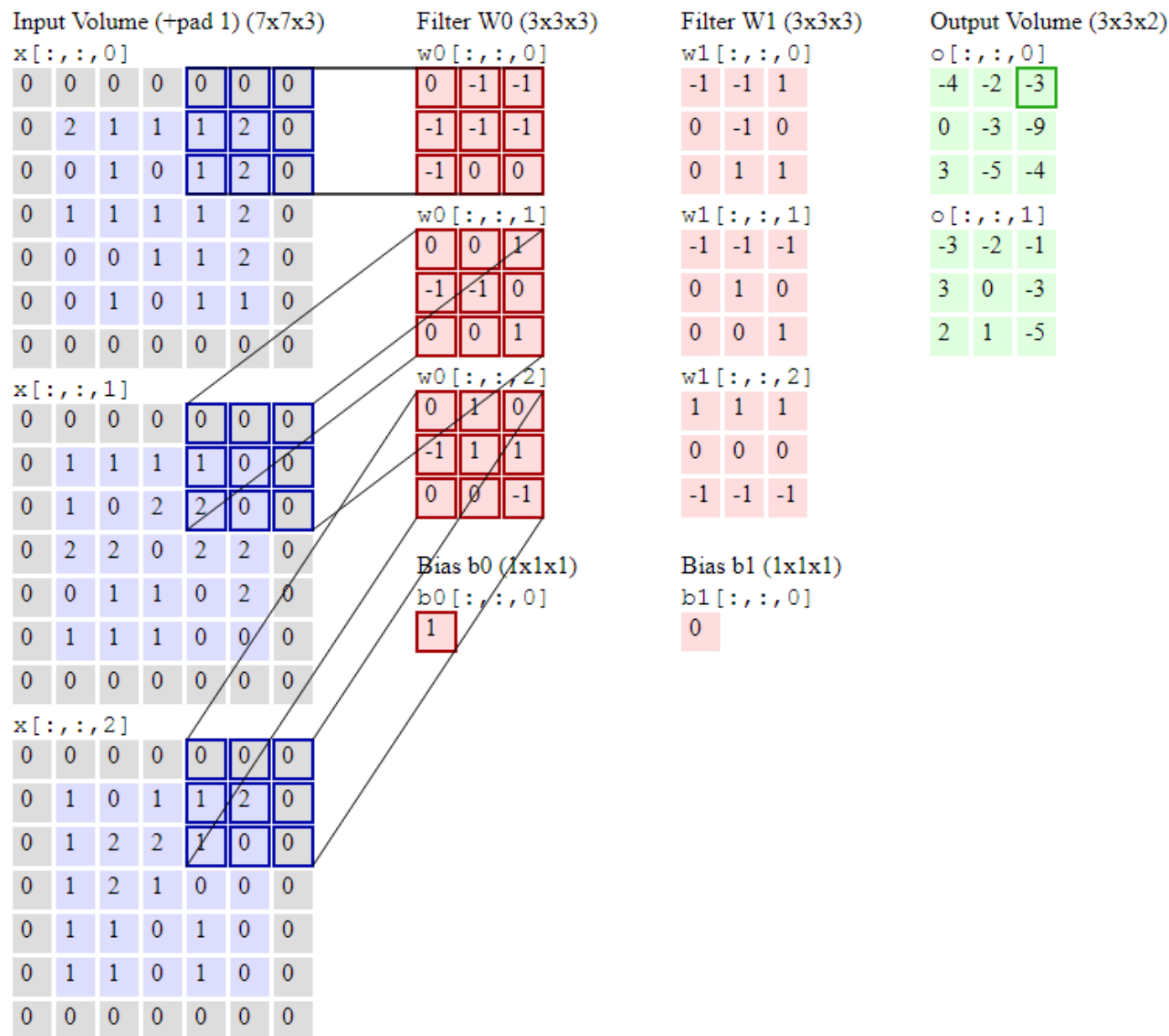
Convolution Operation Demo



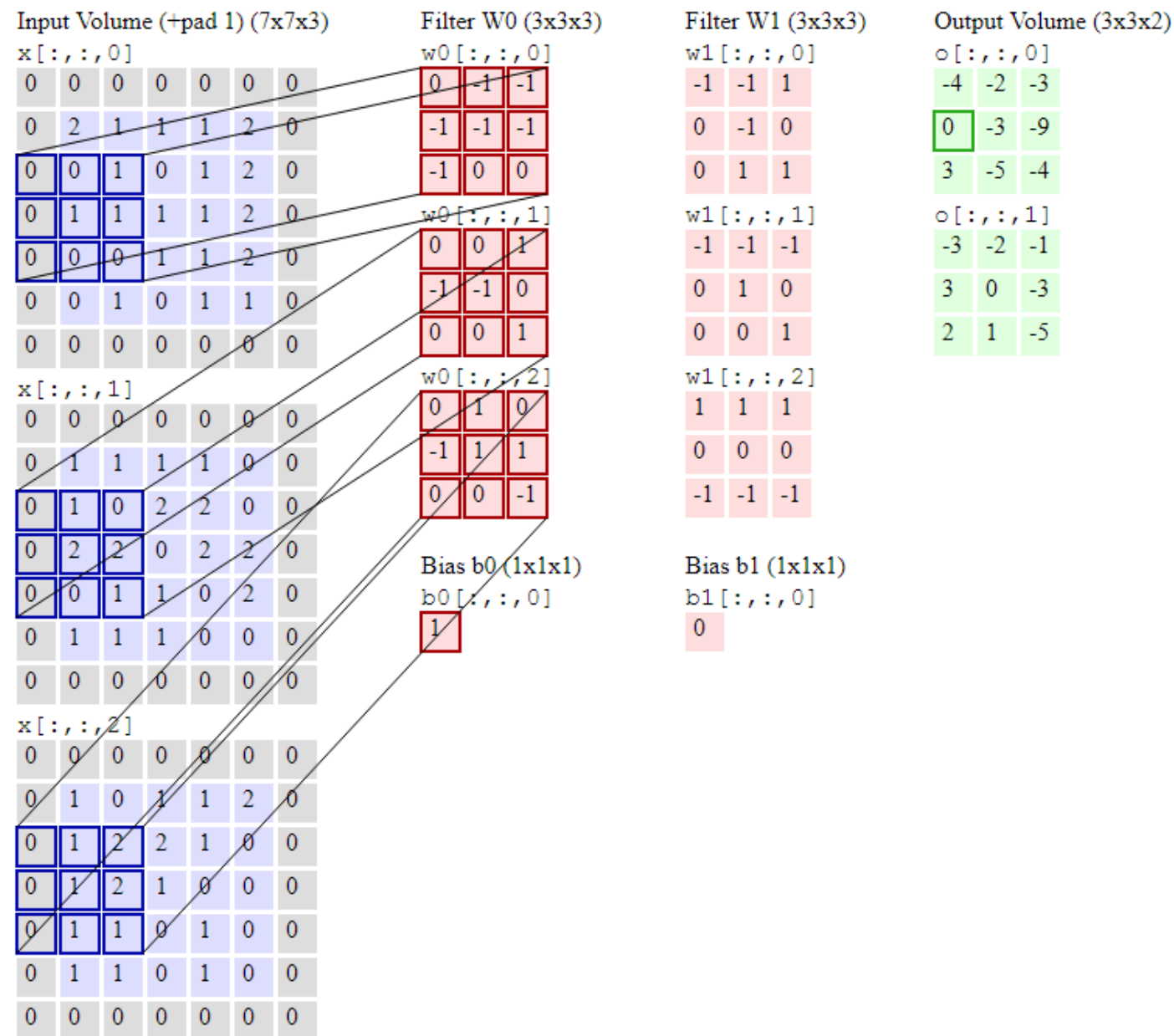
Convolution Operation Demo



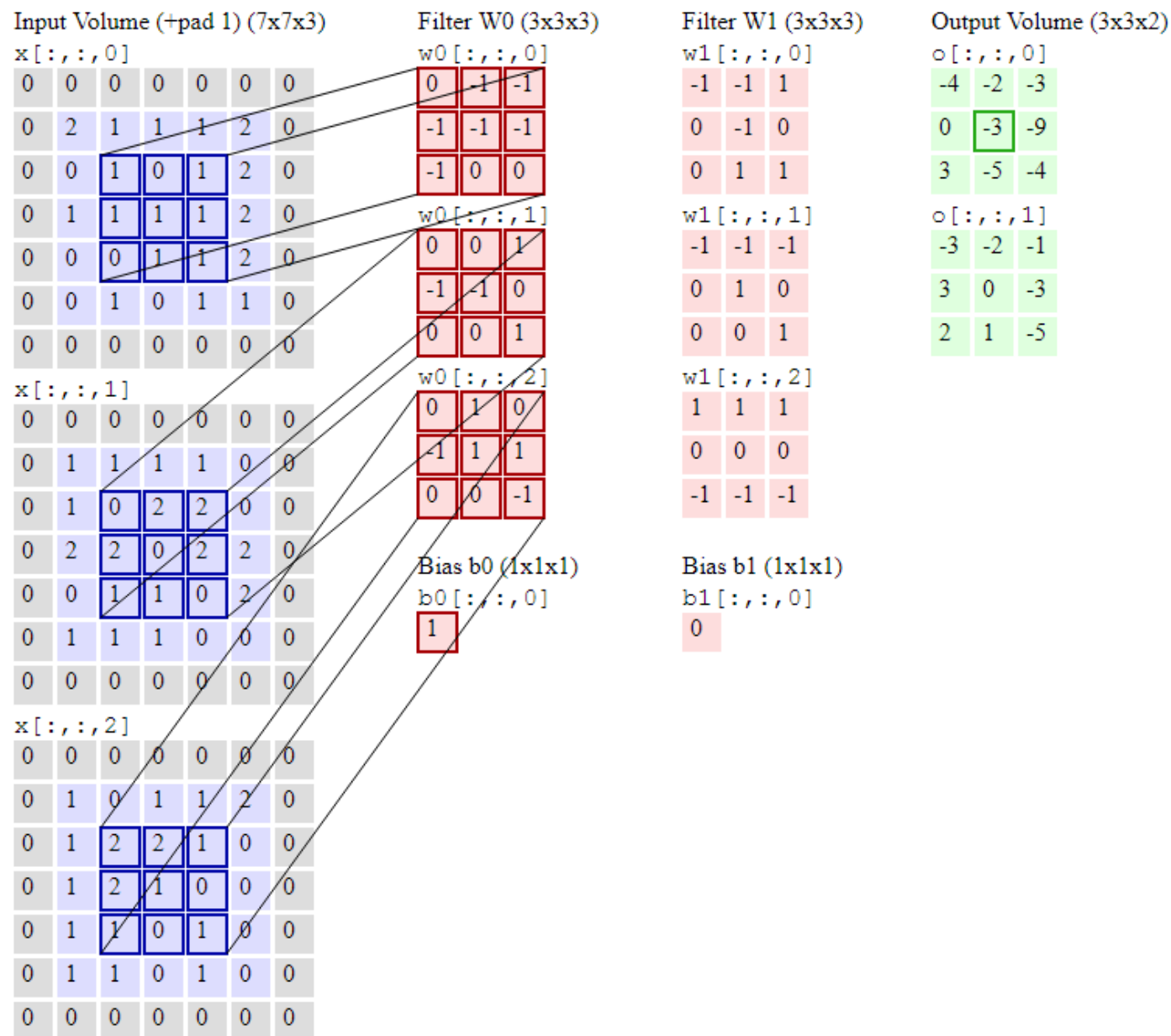
Convolution Operation Demo



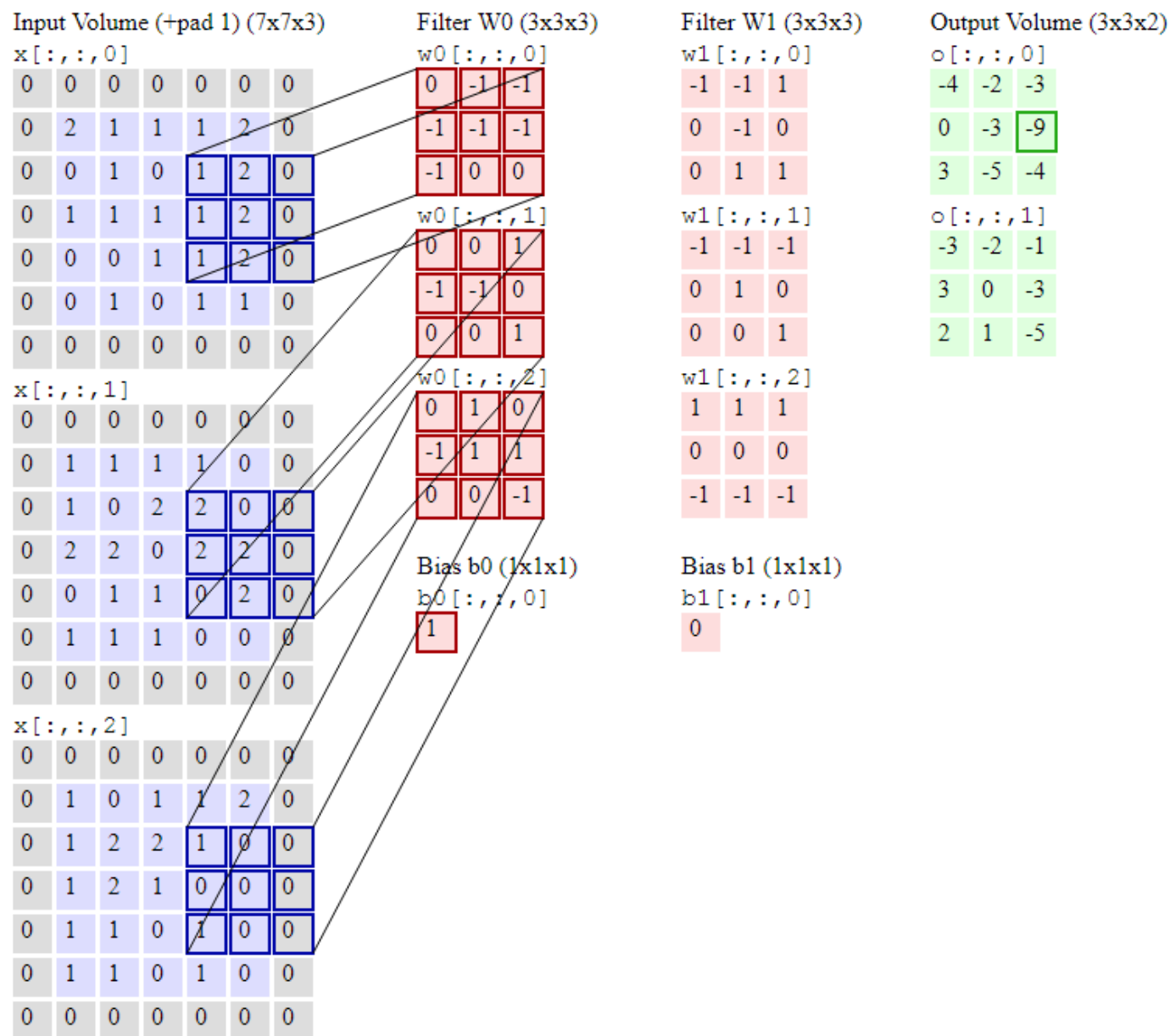
Convolution Operation Demo



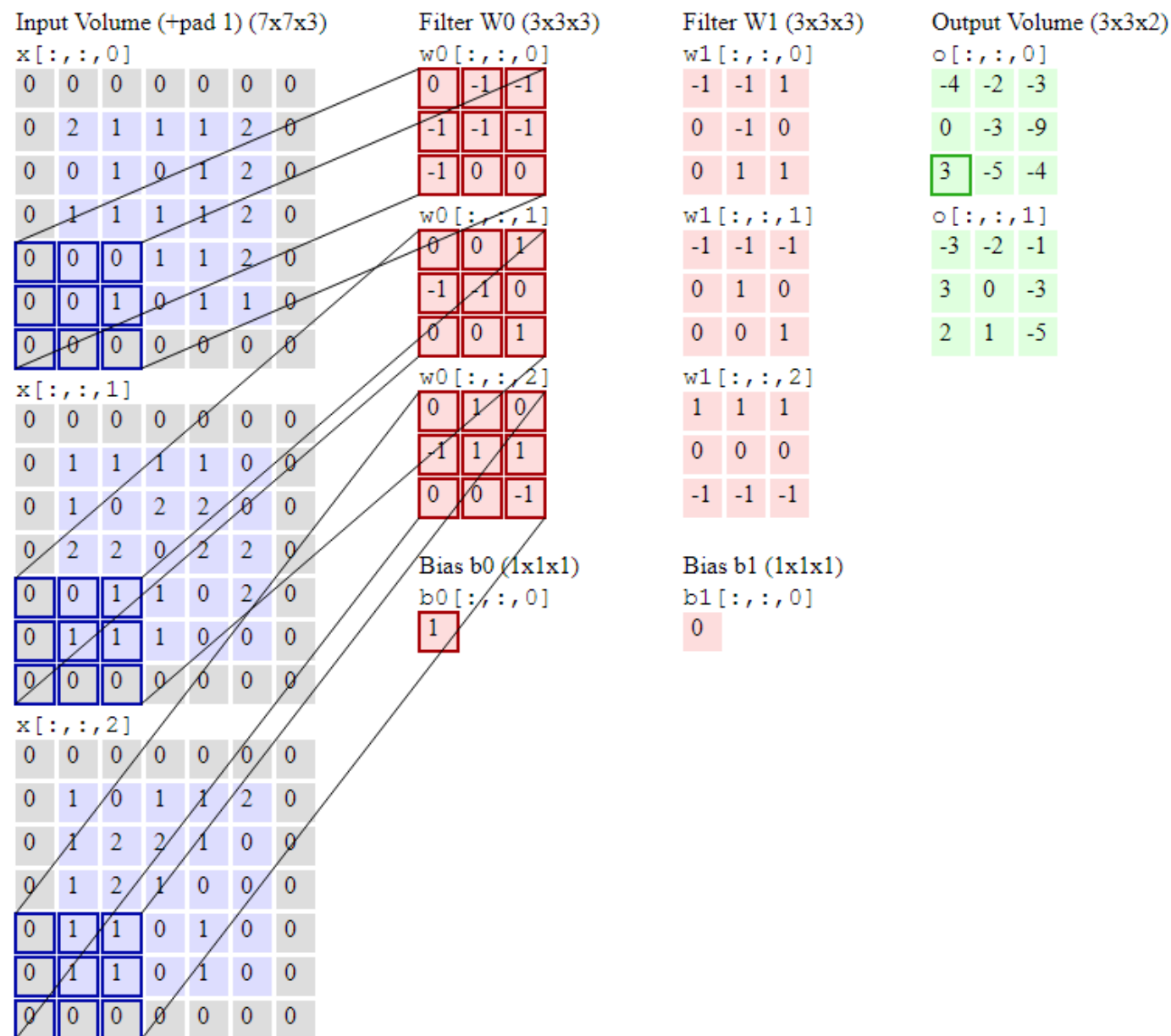
Convolution Operation Demo



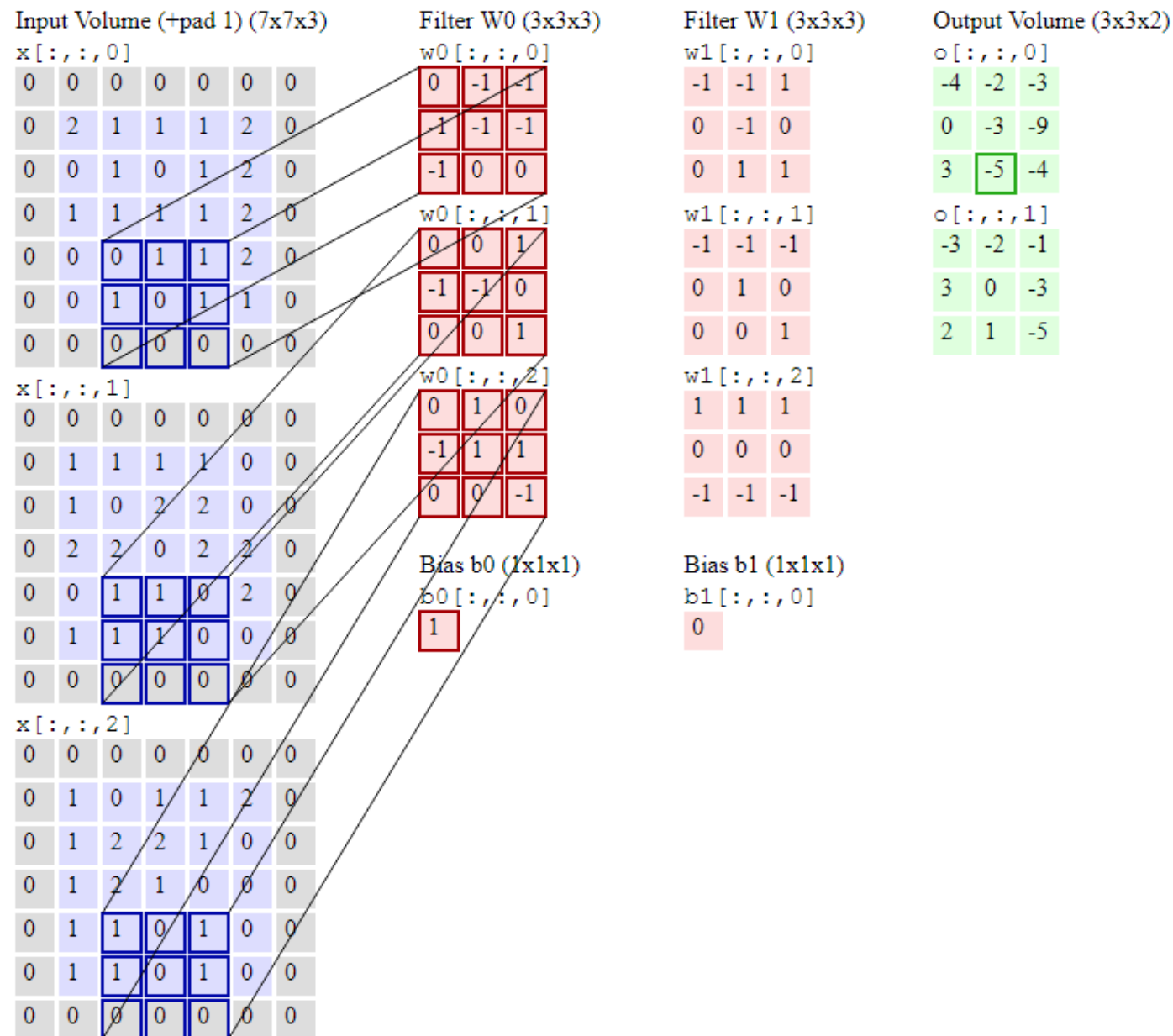
Convolution Operation Demo



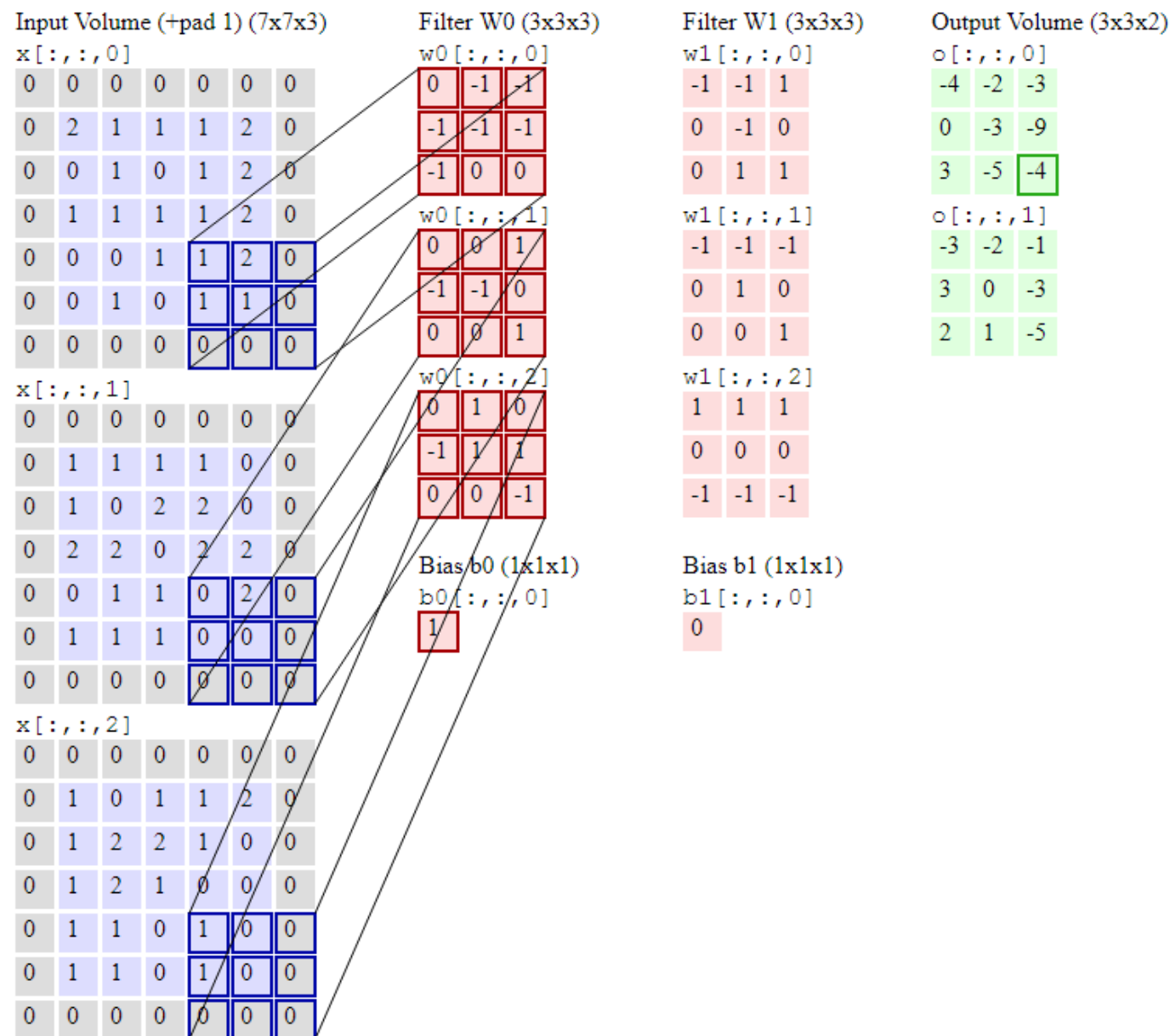
Convolution Operation Demo



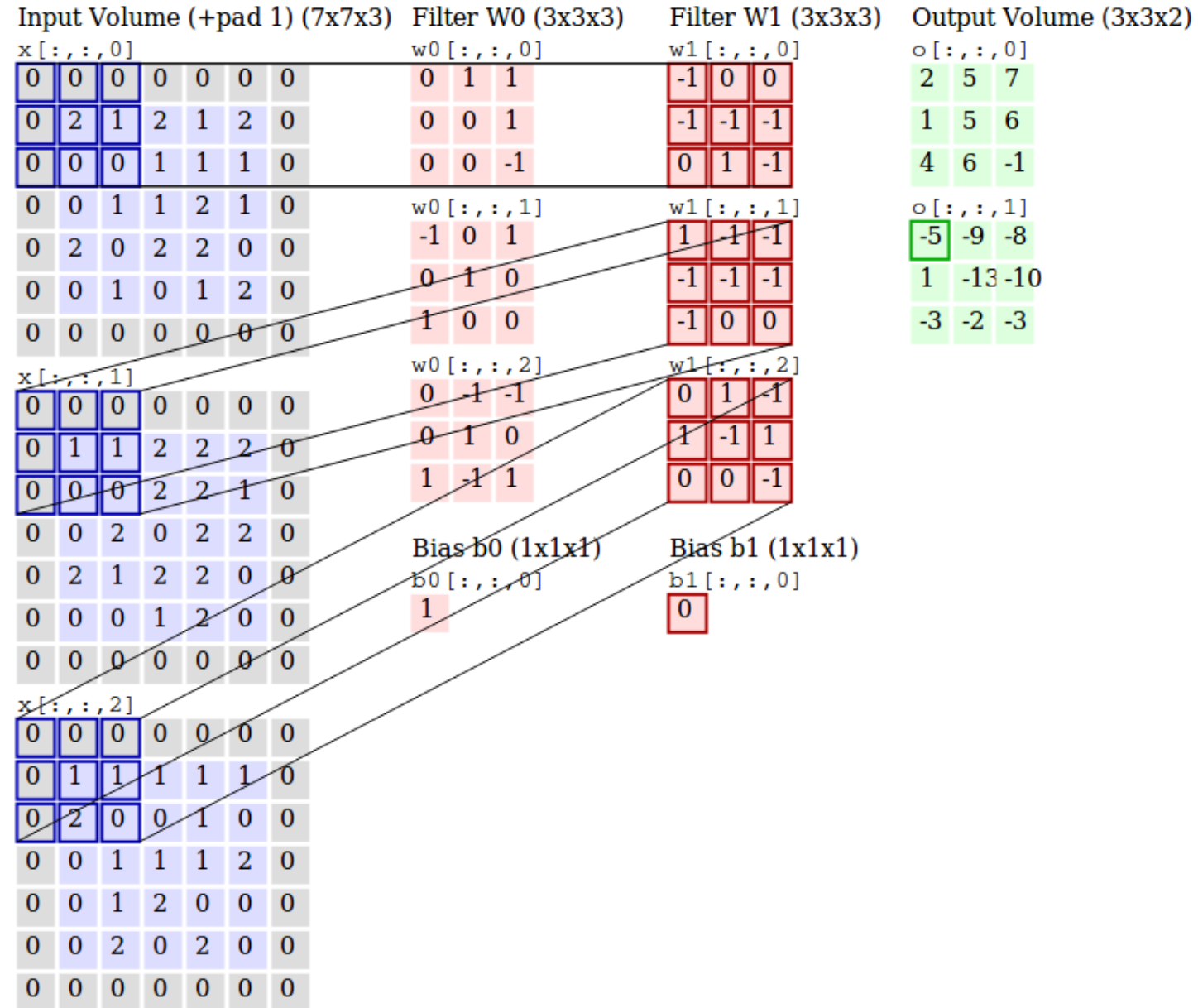
Convolution Operation Demo



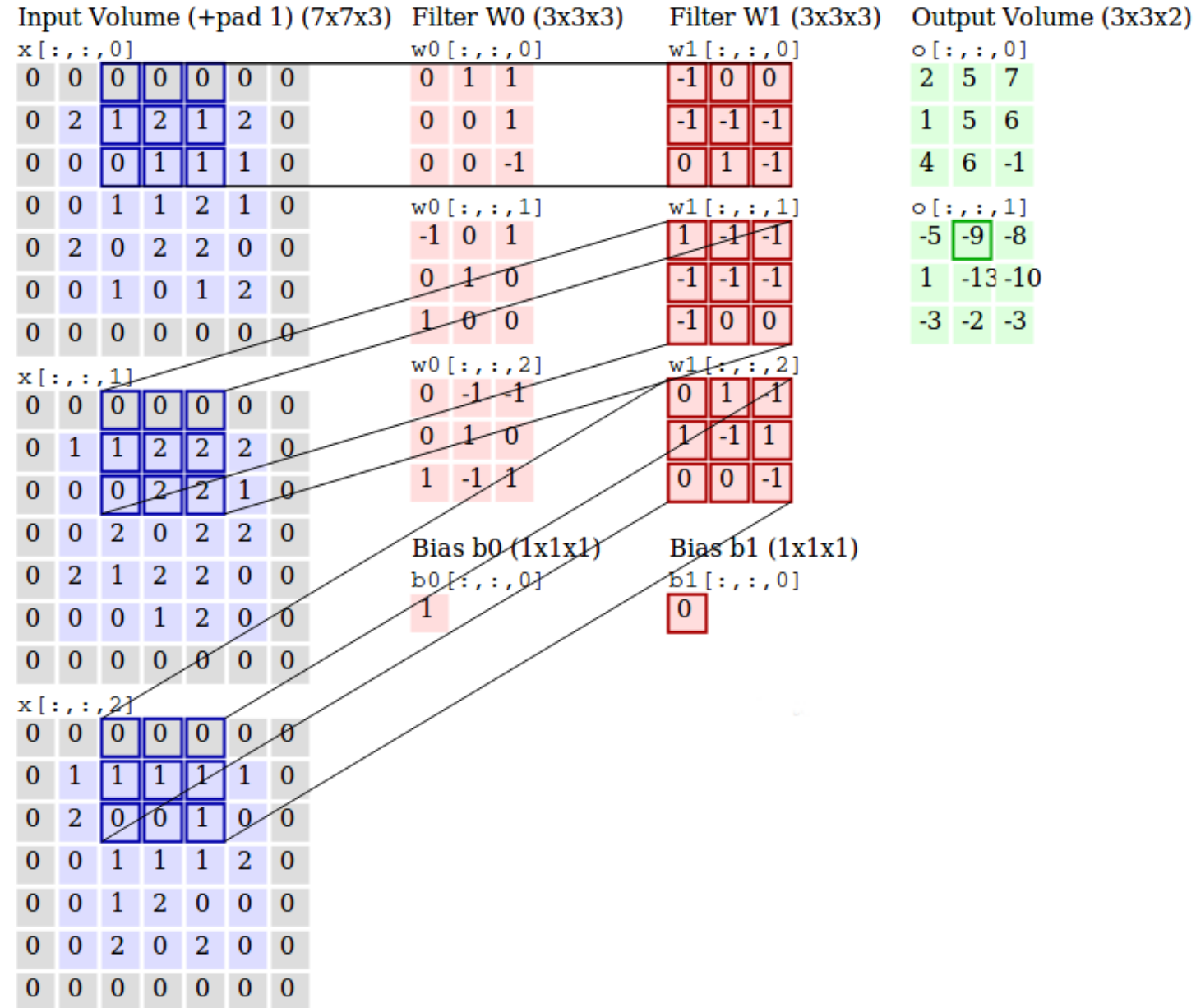
Convolution Operation Demo



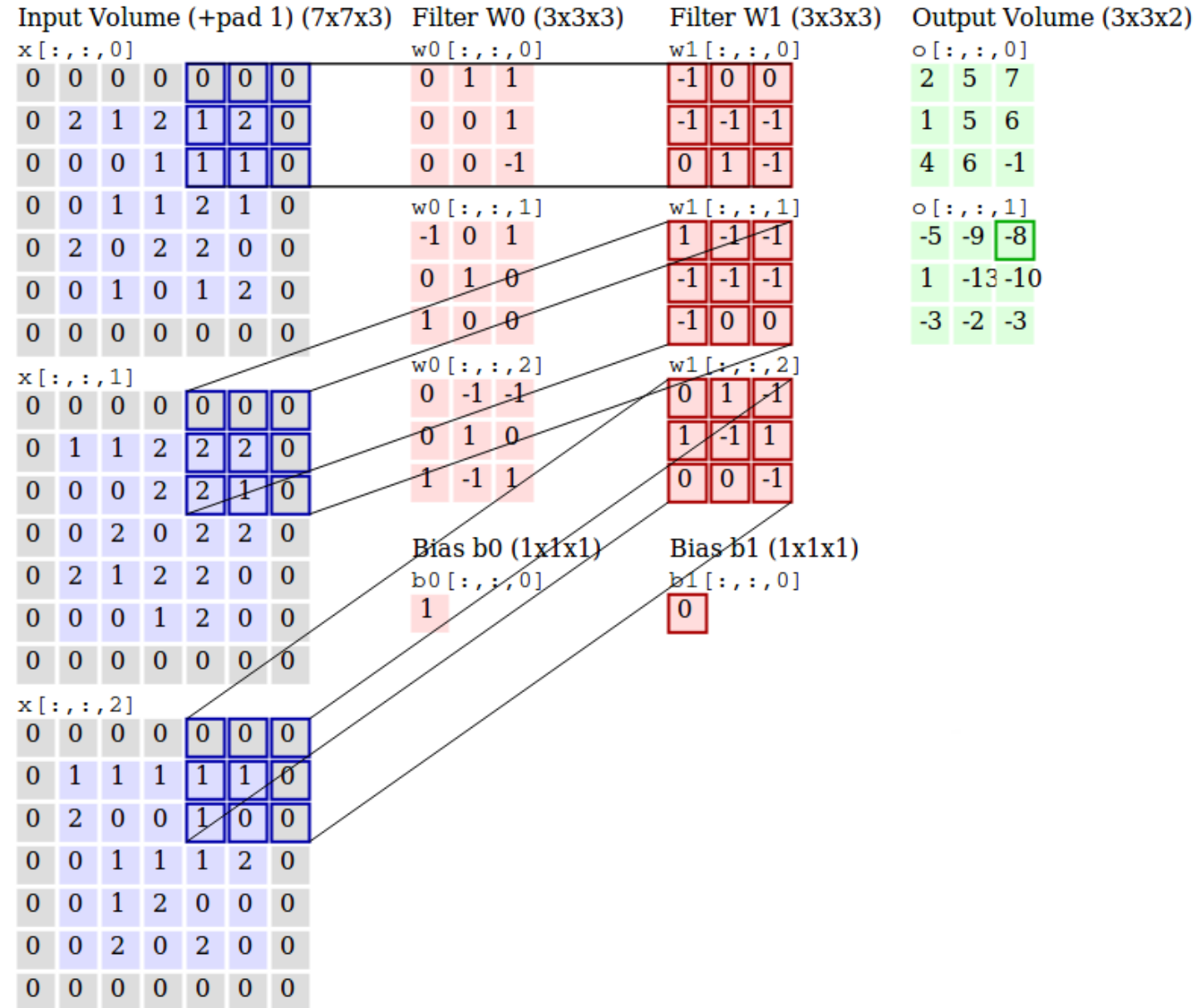
Convolution Operation Demo



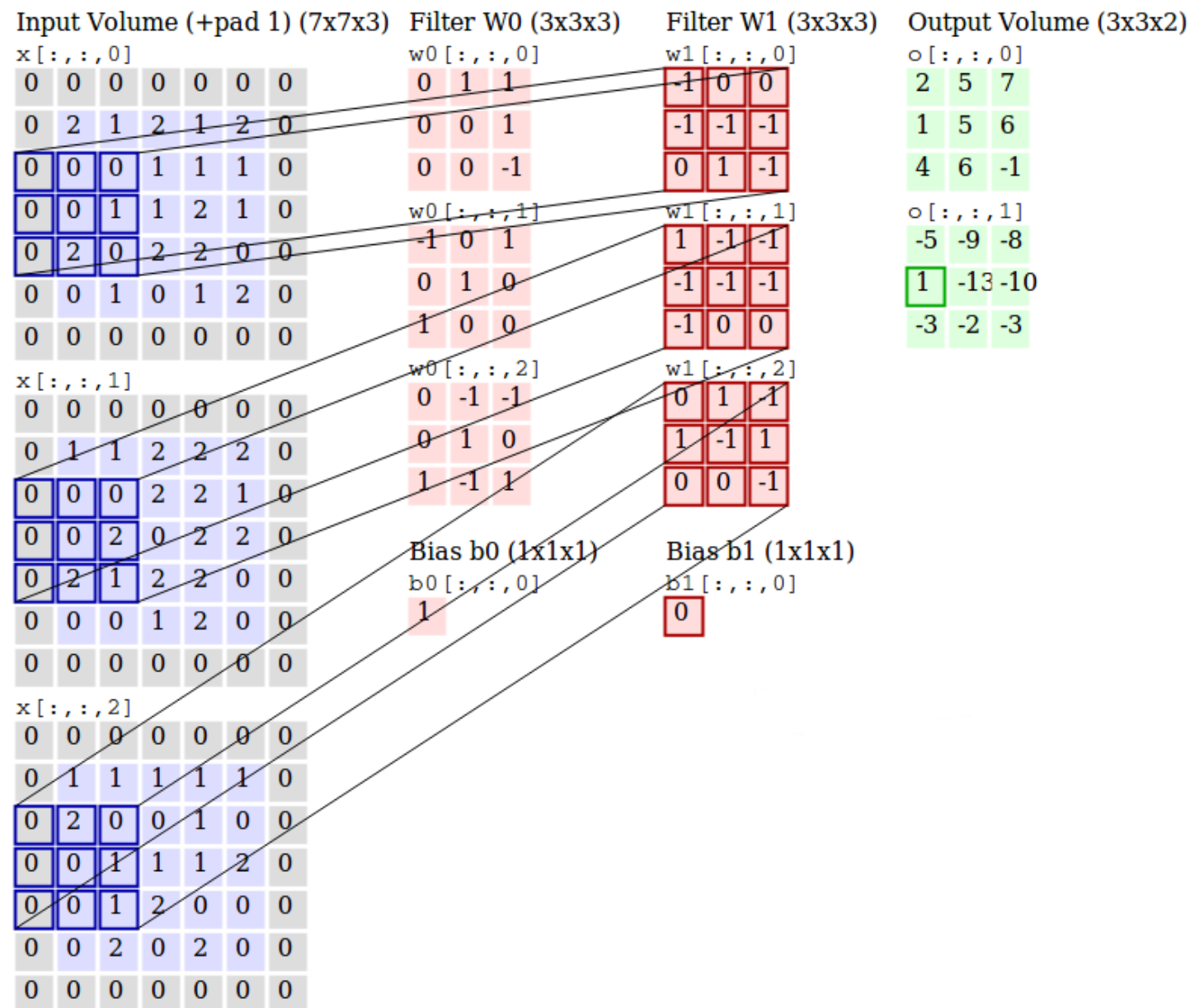
Convolution Operation Demo



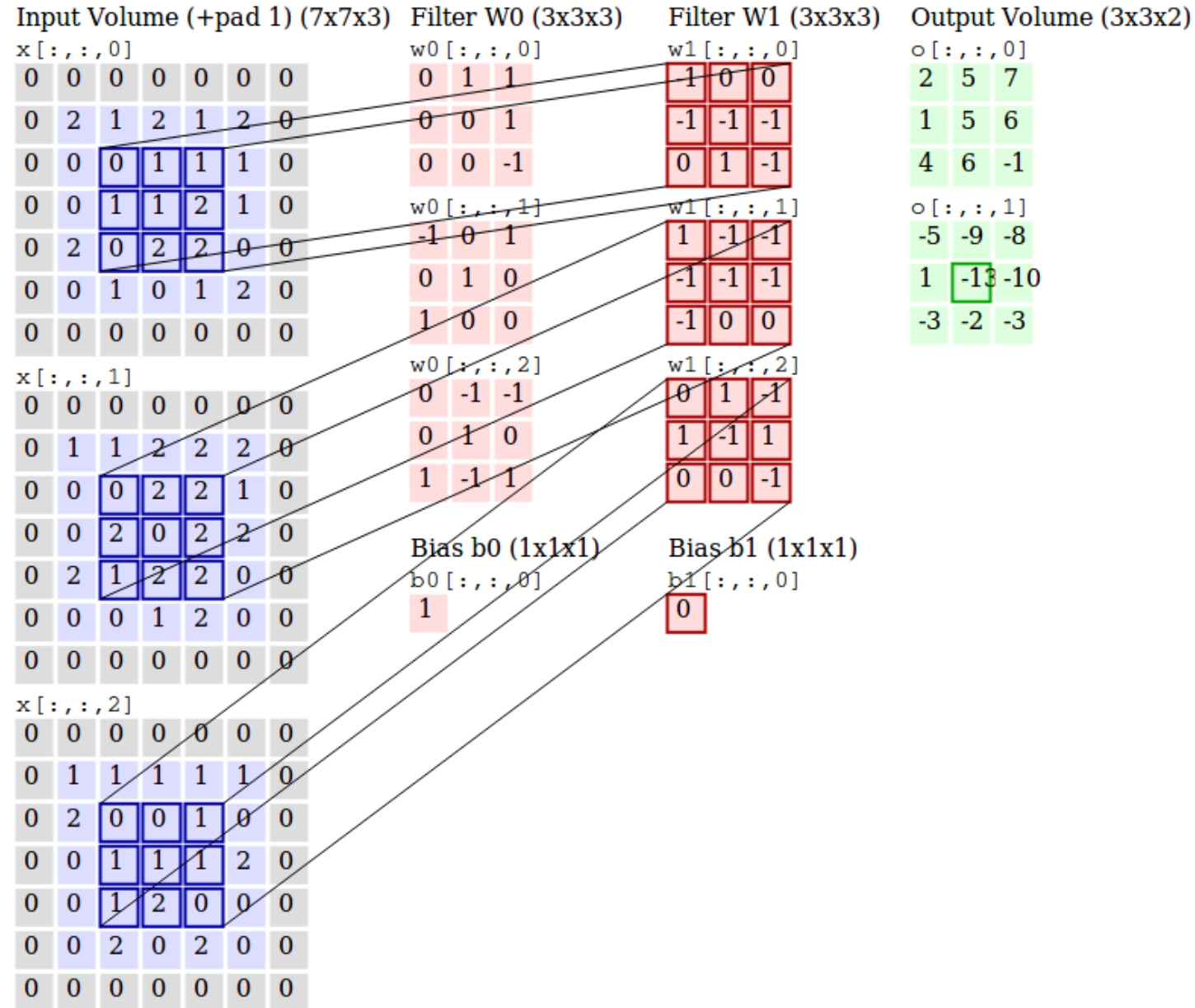
Convolution Operation Demo



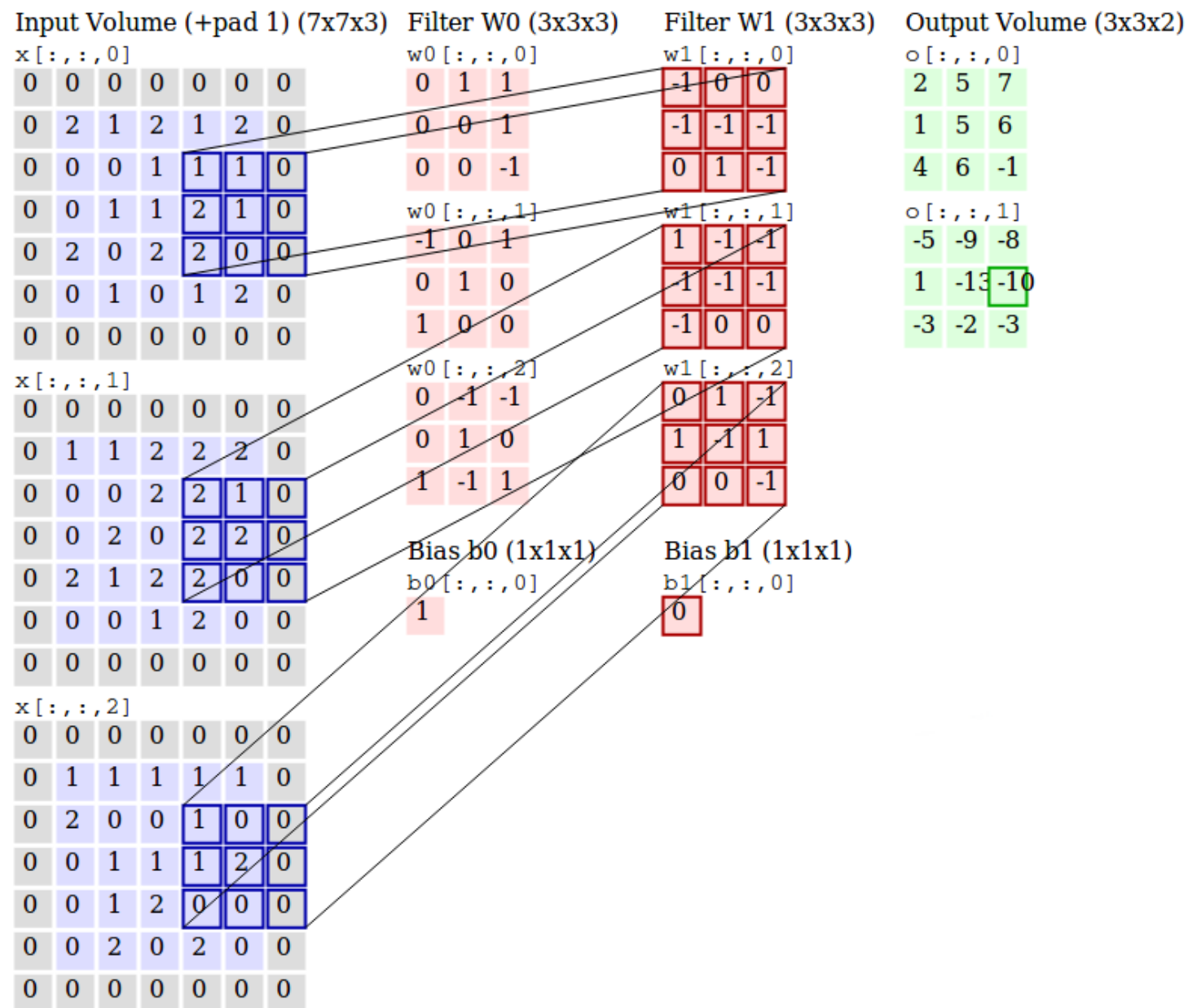
Convolution Operation Demo



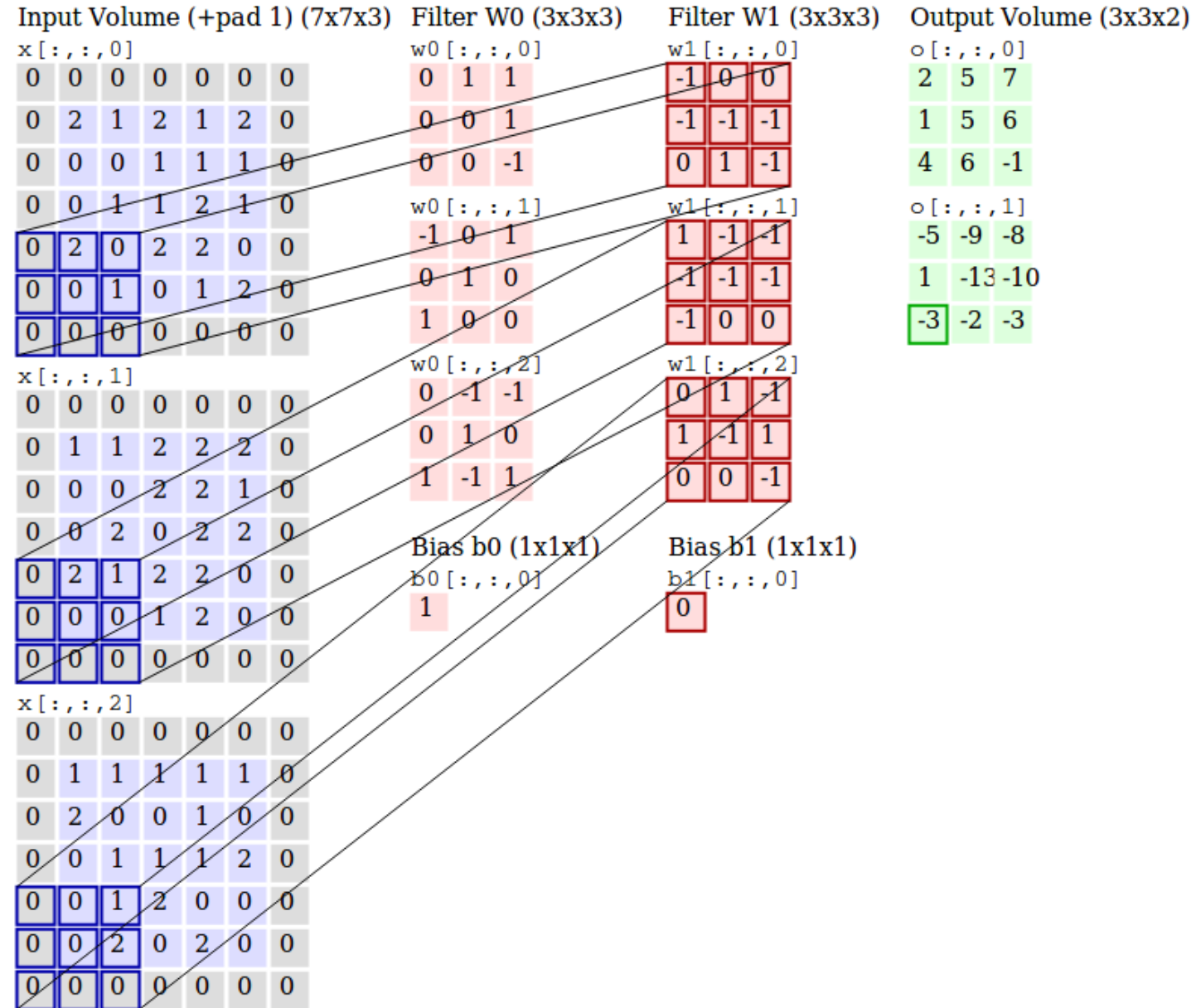
Convolution Operation Demo



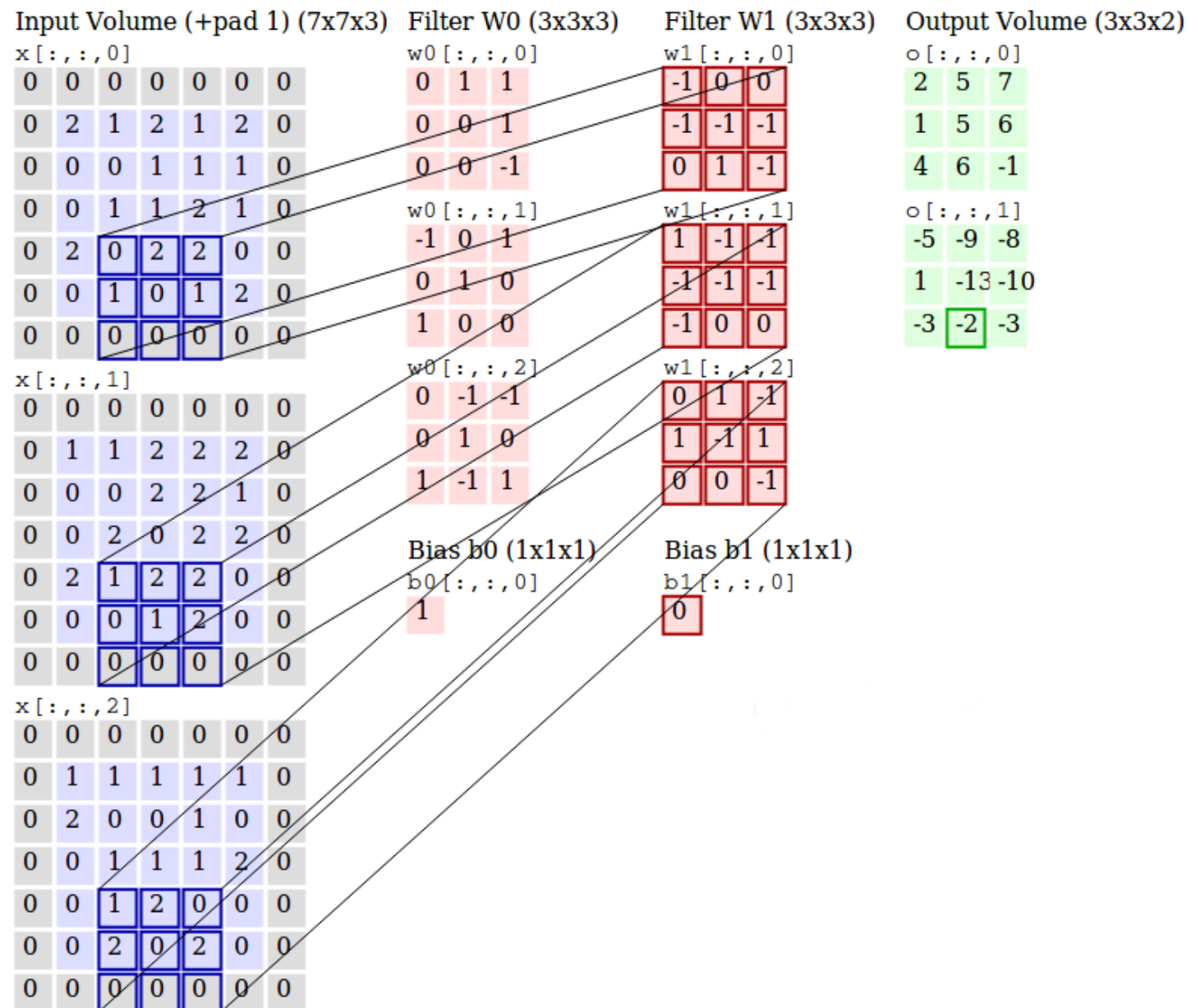
Convolution Operation Demo



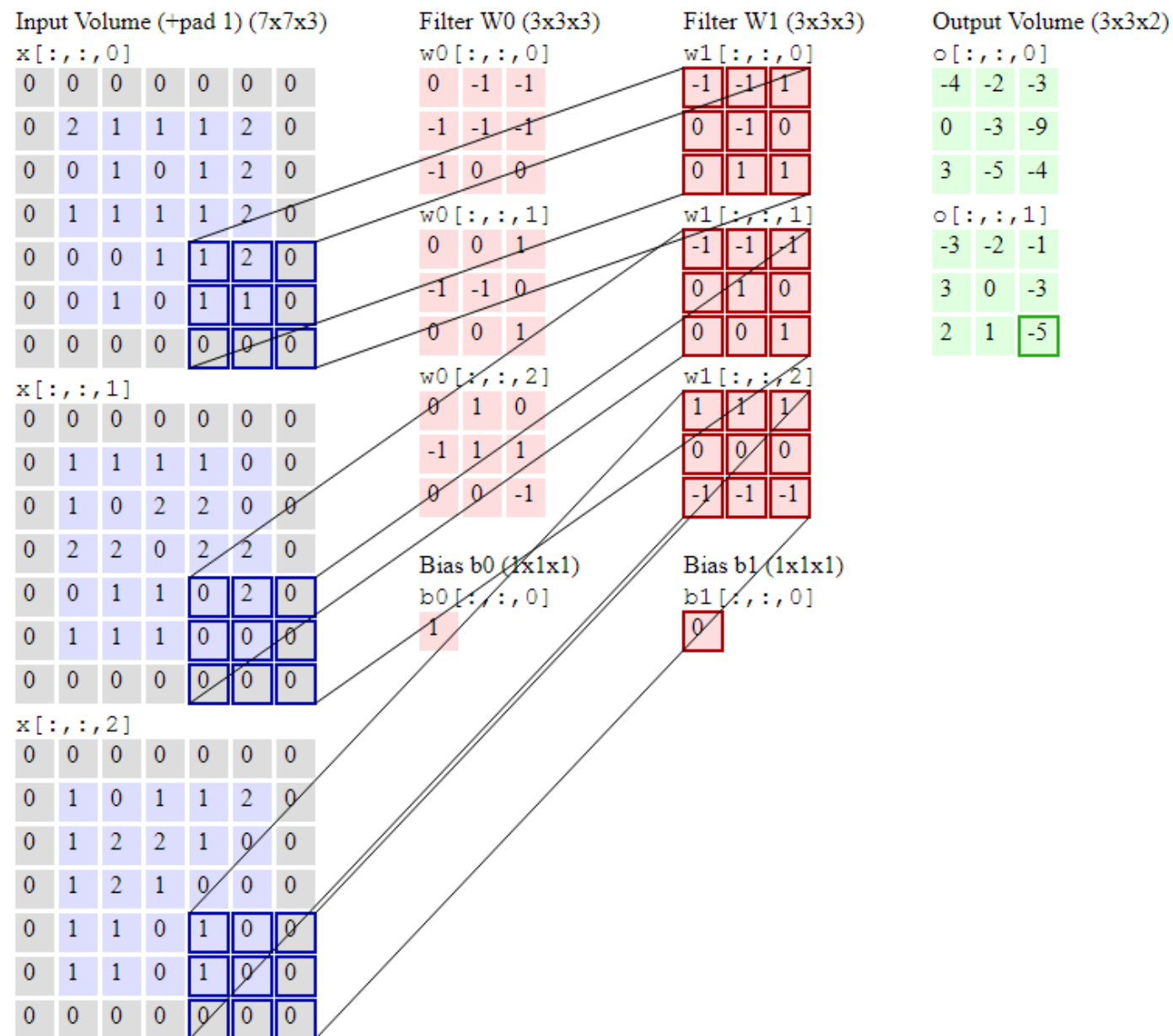
Convolution Operation Demo



Convolution Operation Demo



Convolution Operation Demo



Convolution Layer

Preview

[Zeiler and Fergus 2013]

Visualization of VGG-16 by Lane McIntosh. VGG-16 architecture from [Simonyan and Zisserman 2014].

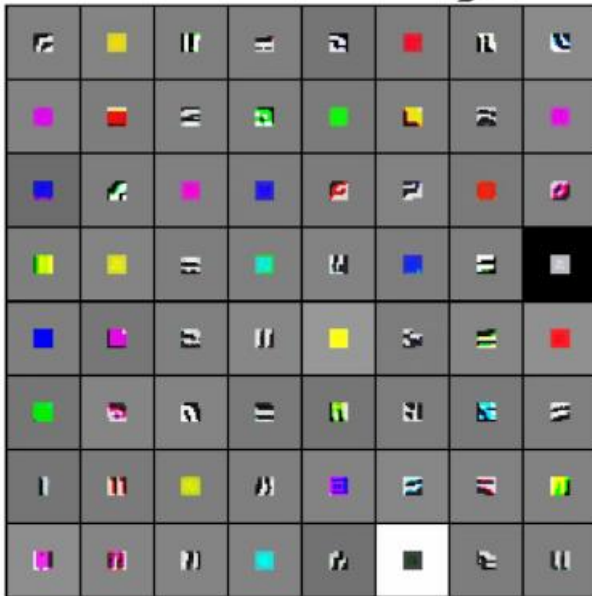


Low-level
features

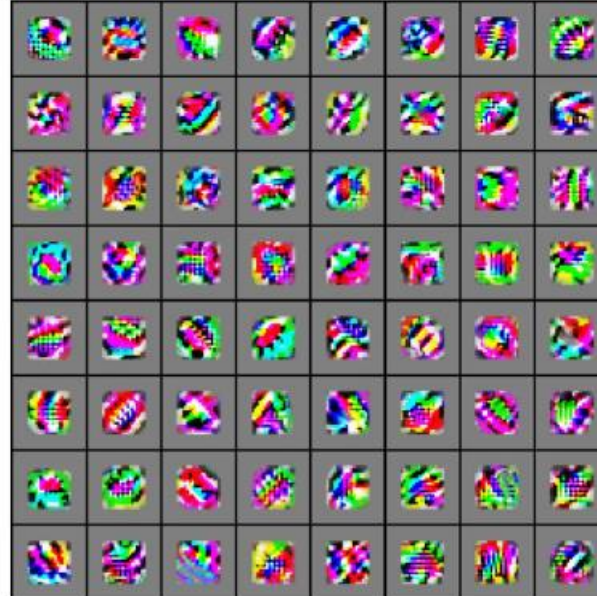
Mid-level
features

High-level
features

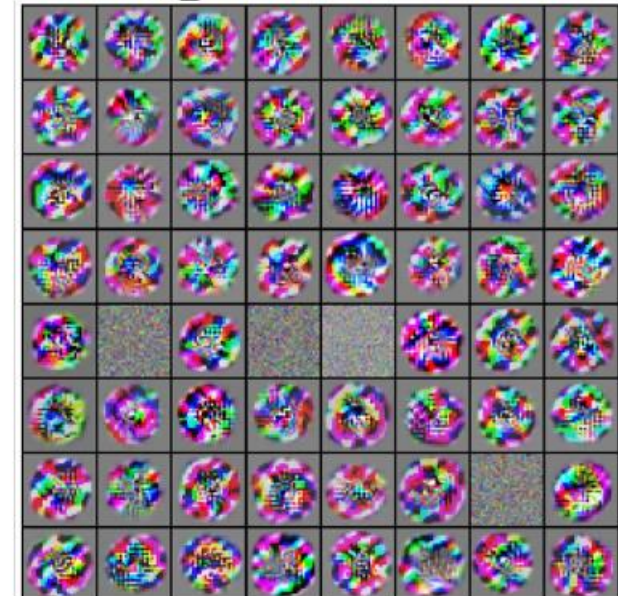
Linearly
separable
classifier



VGG-16 Conv1_1



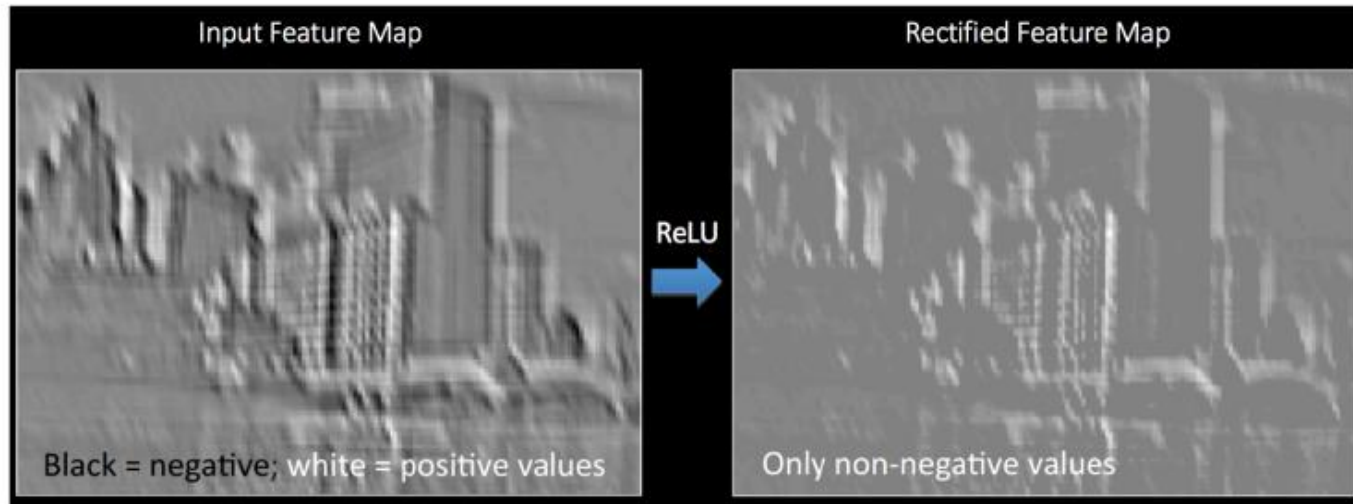
VGG-16 Conv3_2



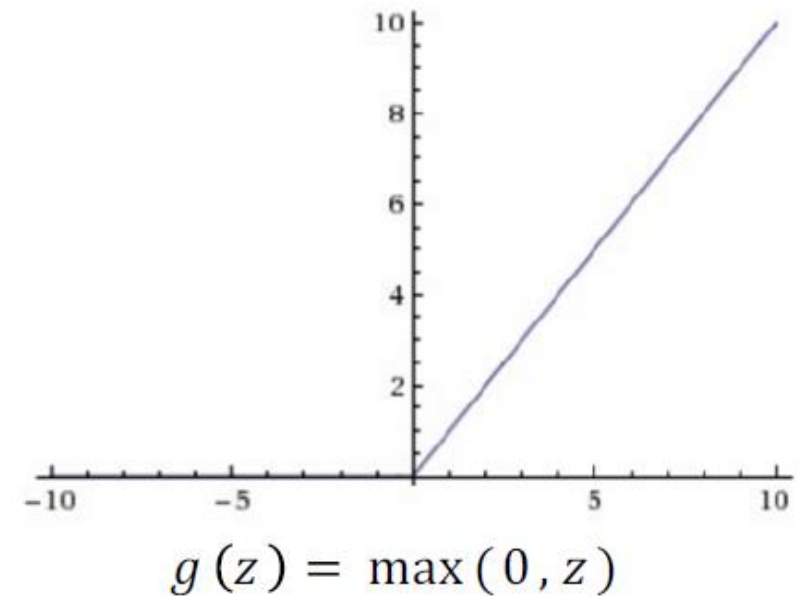
VGG-16 Conv5_3

Introducing Non-Linearity

- Apply after every convolution operation (i.e., after convolutional layers)
- ReLU: pixel-by-pixel operation that replaces all negative values by zero. **Non-linear operation**

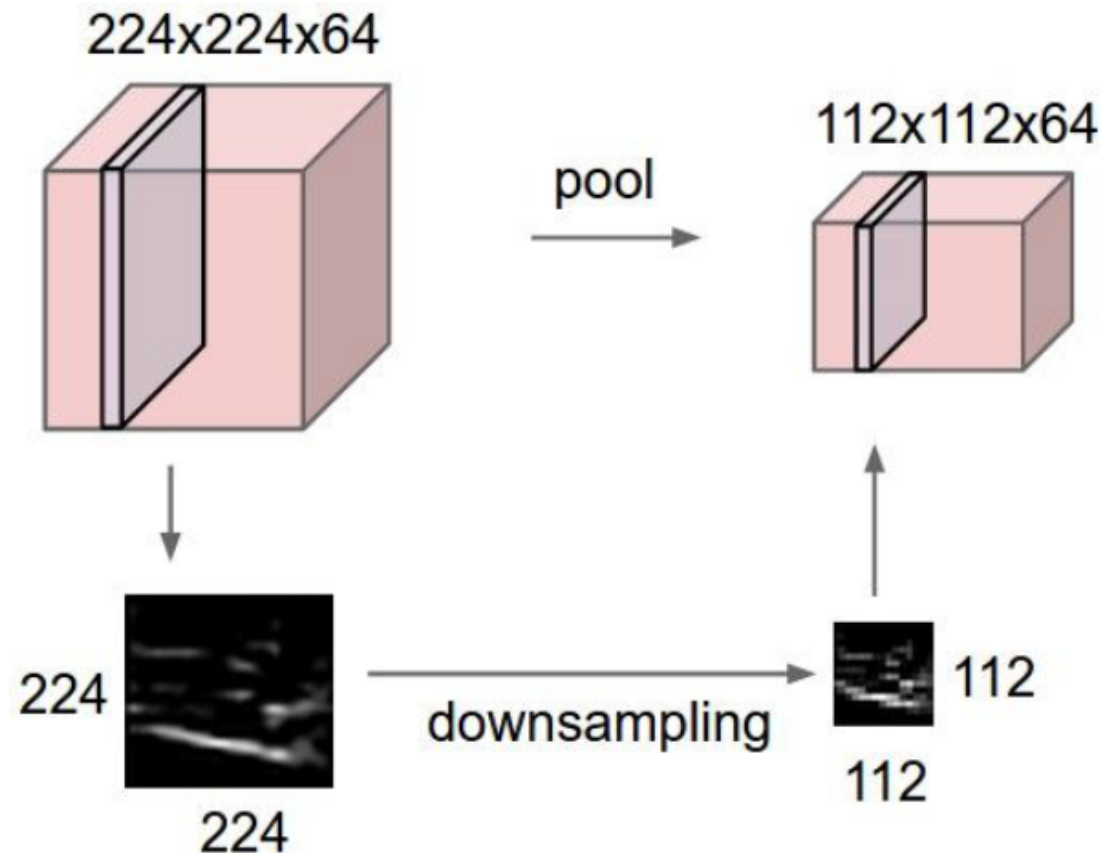


Rectified Linear Unit (ReLU)

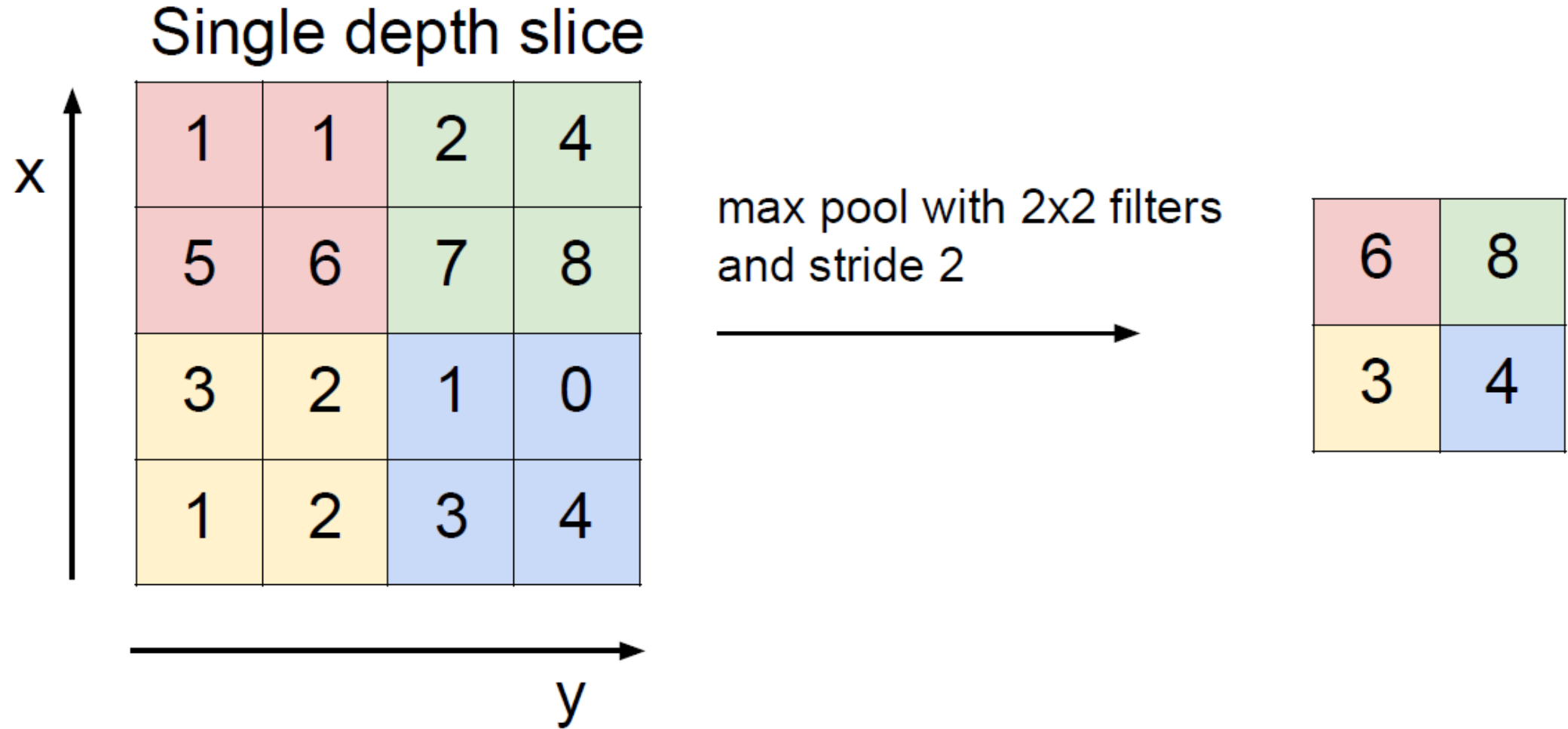


Pooling Layer

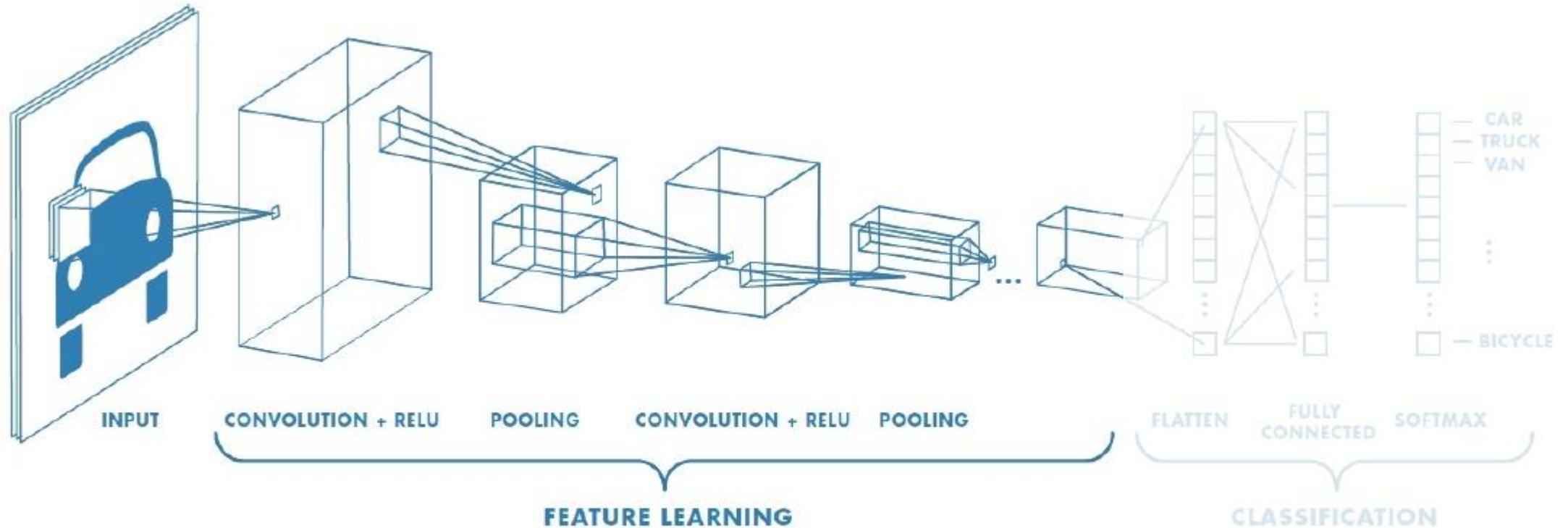
- makes the representations smaller and more manageable
- operates over each activation map independently:



Pooling Layer

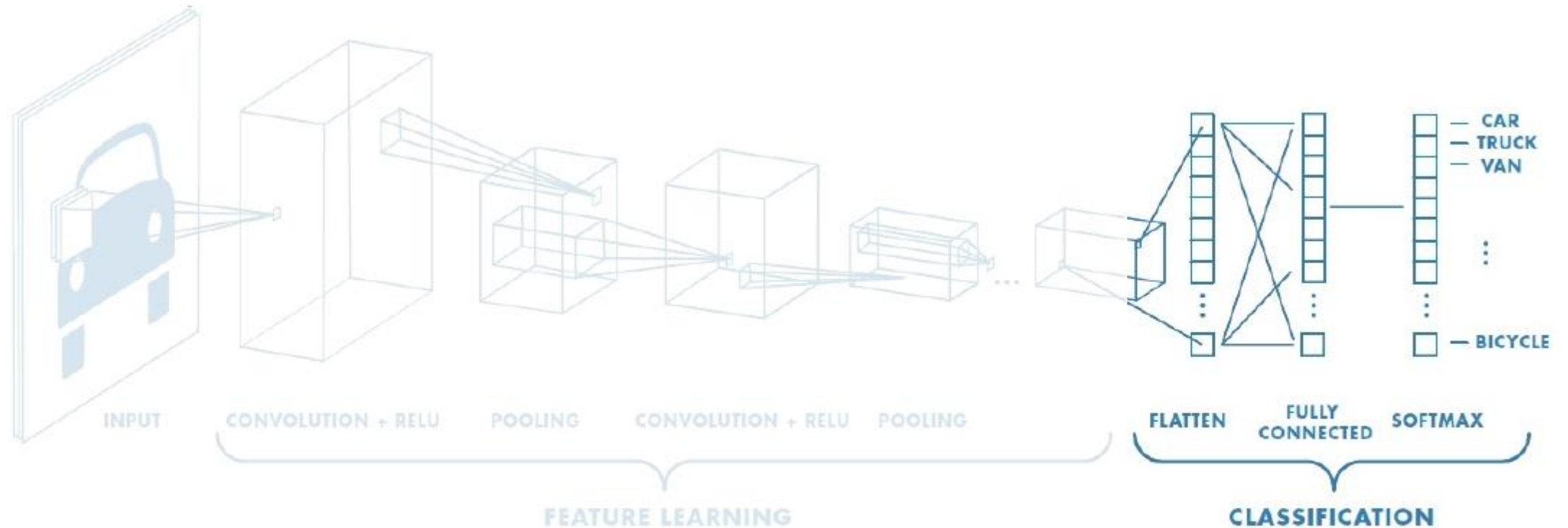


CNNs for Classification: Feature Learning



1. Learn features in input image through **convolution**
2. Introduce **non-linearity** through activation function (real-world data is non-linear!)
3. Reduce dimensionality and preserve spatial invariance with **pooling**

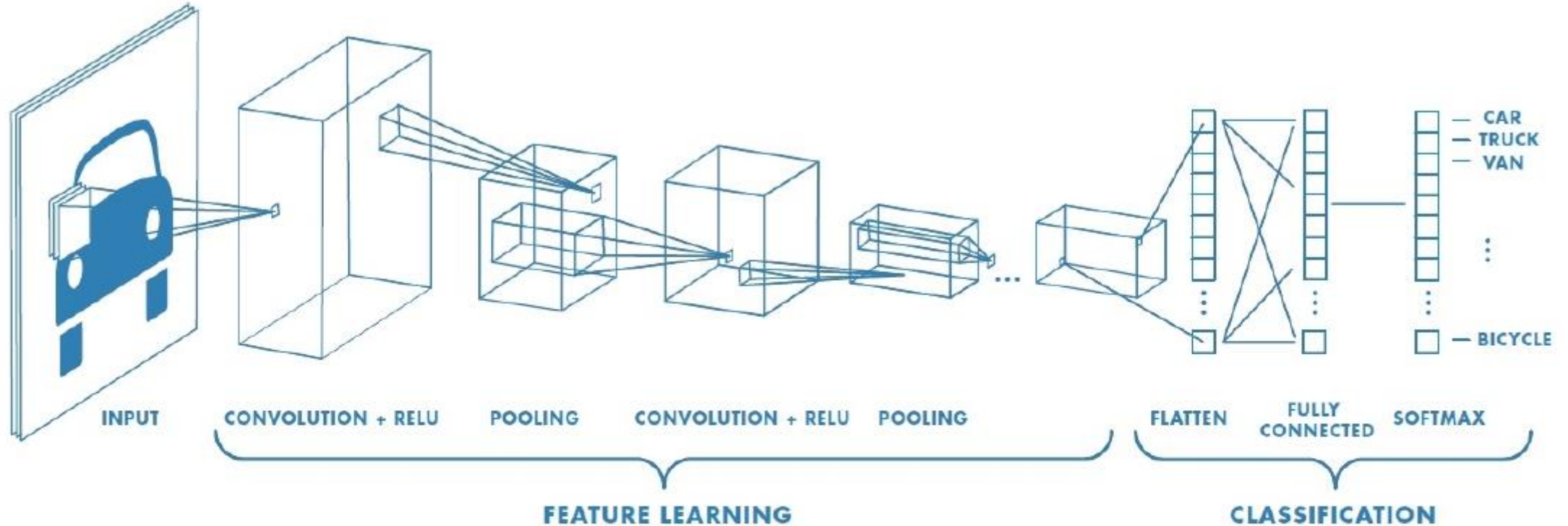
CNNs for Classification: Class Probabilities



- CONV and POOL layers output high-level features of input
- Fully connected layer uses these features for classifying input image
- Express output as **probability** of image belonging to a particular class

$$\text{softmax}(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

CNNs: Training with Backpropagation



Learn weights for convolutional filters and fully connected layers

Backpropagation: cross-entropy loss

$$J(\theta) = \sum_i y^{(i)} \log(\hat{y}^{(i)})$$

Deep Learning Resources

Name	Language	Link	Note
Pylearn2	Python	http://deeplearning.net/software/pylearn2/	A machine learning library built on Theano
Theano	Python	http://deeplearning.net/software/theano/	A python deep learning library
Keras	Python	https://keras.io/	A python deep learning library
Caffe	C++	http://caffe.berkeleyvision.org/	A deep learning framework by Berkeley
Torch	Lua	http://torch.ch/	An open source machine learning framework
Overfeat	Lua	http://cilvr.nyu.edu/doku.php?id=code:start	A convolutional network image processor
Deeplearning4j	Java	http://deeplearning4j.org/	A commercial grade deep learning library
Word2vec	C	https://code.google.com/p/word2vec/	Word embedding framework
GloVe	C	http://nlp.stanford.edu/projects/glove/	Word embedding framework
Doc2vec	C	https://radimrehurek.com/gensim/models/doc2vec.html	Language model for paragraphs and documents
StanfordNLP	Java	http://nlp.stanford.edu/	A deep learning-based NLP package
TensorFlow	Python	http://www.tensorflow.org	A deep learning based python library
PyTorch	Python	https://pytorch.org/	A deep learning based python library

The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A large, solid red speech bubble is centered on the page, pointing downwards. The text "Thank you" is written in white inside the bubble.

Thank you